

Welt 0: Camp

Kapitel A: Setup

Setup

Wir brauchen ein paar **Extensions**, damit die **Inhalte** des Kurses **richtig dargestellt** werden.

Einige **weitere Extensions** können uns helfen, beim **Programmieren** einen **besseren Überblick** und **Flow** zu erreichen.

1. **VS Code**
 2. **Jupyter** (+*Cell Tags, Keymap, Slide Show, Renderer*)
 3. **Python**
 - Path-Variable checken (wenn nicht vorhanden, manuell setzen)
 4. **Tastatur-Shortcuts** checken & neue festlegen (*Run Python File in Dedicated Terminal = Alt+Enter*)
 5. **Zeilennummern** aktivieren (*Notebook: Line Numbers*)
 6. **Extensions**
 - *indent-rainbow*
 - *Material Icon Theme*
 - *VS Code Pets*
 7. **Themes**
 - *One Dark Pro*
-

Ordner-Struktur

Wenn ihr den **gesamten Ordner** im Explorer öffnet, könnt ihr einfach und schnell zwischen verschiedenen Dateien springen.

File → Open Folder → SPEICHERORT

Welt 0: Camp

Kapitel B: Aufbau

Py's Adventure

Generell ist das Material wie folgt aufgebaut:

- **Welten**: gekennzeichnet durch **Zahlen** und einen **Namen**.
 - Die **Welten** behandeln jeweils verschiedene **Themen**, die auf einem **ähnlichen Grad der Komplexität** liegen.
 - **Spätere Welten**, sobald die Grundlagen gelegt sind, fokussieren sich auf eine **bestimmte Thematik oder Disziplin**.
 - Wir sind gerade in **Welt 0**, dem Camp. Hier findest du alle **Grundlagen und Organisatorisches**.
- **Kapitel**: gekennzeichnet durch **Buchstaben** und ihr **Thema**.
 - **Jedes Kapitel** behandelt ein **bestimmtes Element** von Python.
 - Jedes Kapitel hat **Theorie**, **Challenges** und einen **Playground**.
 - **Theorie**: Hier behandeln wir zusammen die **Grundlagen und Anwendung** von verschiedenen Python-Aspekten.
 - **Challenges**: Hier kannst du **selbst Hand anlegen** und die Inhalte direkt **anwenden**.
 - **Playground**: Hier kannst du die **Inhalte** des Kapitels **frei ausprobieren** und **manchmal** gibt es **etwas extra Code** dazu.
- **Skill-Checks**: in diesen Ordnern findest du verschiedene **.py Dateien**, in denen **Aufgaben mit diversen Schwierigkeitsgraden** zusammenkommen, an denen du deine **Skills** aus dieser Welt **testen** kannst.

Welt 0: Camp

Kapitel C: Python Basics

Worum geht's hier?

1. Was ist Python?

- **Programmiersprache** (wie z.B. *Java*, *C++*, etc.)
- **Open Source** (*Github*)
- Entwickelt von **Guido van Rossum** (90er), viele Contributor
- Name ist eine **Anlehnung an Monty Python**, aber die **Schlange** ist oft **inoffizielles Maskottchen**
- Sowohl **Compiler** als auch **Interpreter Sprache** (*mehr Infos*)
- **Wichtig**: Python ab **3.0** ist eine beinahe **komplett neue Sprache/Umgebung** im Vergleich zu 2.x Versionen

1. Wofür benutzt man Python?

- Oft in **Data Science**, bei **großen Datenmengen**
- Sehr gut und schnell in **mathematischen Operationen**
- **Weites**, und auch **tiefes, Einsatzgebiet** (*Jack of All Trades, Master of Some*)

1. Was gehört zu Python?

- Python Dokumentation
- Viele und flexible **In-Built**s
- **Unzählige** externe (open source) **Module & Pakete** (PyPi)

1. Warum lernen wir Python?

- **Einfachheit**, sehr gut um den **Einstieg** ins Programmieren zu lernen
- **Nutzerfreundlich & nachsichtig**
- Eine der/die **gefragteste Programmiersprache**, Tendenz steigend

TIOBE Index (Stand: 22.09.25):

{8E5189D7-906E-44FF-8968-6C8DDA8F0E03}.png

**Also Python ist eine Interpreter Sprache
entwickelt von Guido van Rossum, der
Name ist eine Anlehnung an Monty
Python**

Welt 0: Camp

Kapitel D: Kommentare

Kommentare im Code

1. Was ist ein Kommentar?

Wird vom **Computer** (Compiler/Interpreter) **ignoriert**.

2. Warum sollte ich Kommentare nutzen?

Sie helfen, **Struktur** für dich (und andere) in den Code zu bringen.

Sie können verschieden genutzt werden, z.B. um **zusammenzufassen**, was ein bestimmter Teil des Codes tut oder wann er ausgeführt wird.

Besonders **beim Lernen & Ausprobieren** hilfreich.

ABER: **guter Code** zeichnet sich dadurch aus, dass er **auch ohne Kommentare** gut lesbar & klar verständlich ist.

```
# Ich bin ein Kommentar.
### Ich bin auch ein Kommentar.
##Ich auch!
```

```
"""
```

```
Ich bin ein Kommentar,
```

```
' \nIch bin ein Kommentar,\nder sogar über mehrere Zeilen geht.\nAber ich werde jetzt grün angezeigt?\n'
```

Gibt in der Konsole aus, was in den Klammern steht.
Um direkt **Text** auszugeben, wird dieser in **"Anführungszeichen"** gesetzt.

```
print("hallo welt")
print("Hello\nWorld")
print("Hello", "World")
```

```
hallo welt
Hello
World
```

Challenge 1

Benutze die *print()* Funktion, um **diese Ausgabe** zu **erhalten** (mit Zeilenumbrüchen):

```
Die Python print() Funktion
wird wie folgt deklariert:
print('Das hier wird ausgegeben.')
```

```
print("Die Python print() Funktion")
print("wird wie folgt deklariert:")
print("Das hier wird ausgegeben.")
```

```
Die Python print() Funktionwird wie folgt deklariert:
Das hier wird ausgegeben.
```

Lösung

```
print("Die Python print() Funktion\nwird wie folgt deklariert:\nprint('Das\nhier wird ausgegeben.')
```

```
Die Python print() Funktion
wird wie folgt deklariert:
print('Das hier wird ausgegeben.')
```

print(): end & sep

Parameter	Beschreibung	Default
<i>end="X"</i>	<i>X</i> wird am Ende angehängen.	Zeilenumbruch: <i>\n</i>
<i>sep:"X"</i>	<i>X</i> wird zwischen jedem Element eingesetzt.	Leerzeichen: <i>" "</i>

```
print("Hello", "World", end="\n") #normwerte
print("This is", "Python", sep=' ') #normwerte
print("Hello", "World", end="###")
print("This is", "Python", sep='---')
```

```
Hello World
This is Python
Hello World###This is---Python
```

Challenge 2

Nutze *sep* und *end* zusammen mit dem *print()* Befehl, um die **Ausgabe genau so** darzustellen, **wie hier** (ohne die **Texte** und *print()* Befehle zu **verändern**):

```
Wir***lernen***gerade...Python
```

```
print("Wir","lernen", "gerade")
print("Python")
```

```
Wir *** lernen gerade
Python
```

Lösung

```
print("Wir","lernen", "gerade",sep='***',end="...")
print("Python")
```

```
Wir***lernen***gerade...Python
```

Playground

```
print("Hello", "World")  # Hello World

# Sep
# Standard Separator ist das Leerzeichen - sep=" "
print(1, 2, 3)           # 1 2 3
print(1, 2, 3, sep='- ') # 1-2-3
print(7, 8, 9, sep='')   # 789

# End
# Standard Ende ist der Zeilenumbruch - end="\n"
print(1, end=' ')
print(2)  # 1 2

print("Hello\nWorld")
# Hello
# World

# Hier hast du Platz, dich auszutoben
```

Welt 1: Gestrandet

Kapitel B: Input

input()

input() öffnet einen **Prompt** an den Nutzer und **wartet**, bis die **Eingabe** bestätigt ist. Es gibt **immer** einen **String** zurück.

```
input()
```

Welt 1: Gestrandet

Kapitel C: String Basics

String Basics

Es gibt **verschiedene Möglichkeiten Strings** zu **bauen** und mit ihnen zu **interagieren**:

Element	Beschreibung
"STRING"	grenzt einen String ein .
'STRING'	grenzt einen String ein .
\X	markiert X als literal (wird als Text verarbeitet).
R"STRING"	markiert alles als literal (außer schließende Anführungszeichen).
"STRING", "STRING"	fügt mehrere STRINGS automatisch zusammen .

Tupel sind immutable (unveränderbar).

```
"text"
'text'
"Hi I'm"
print('Hi I\'m here') # Escaping durch das: \
print(R"sghsgjgjf\' gfjfg' #gjh'") #Escaper wird mitausgegeben

Hi I'm here
sghsgjgjf\' gfjfg' #gjh'
```

Challenge 3

Korrigiere die Fehler im Code, damit alle **print()** Befehle richtig ausgeführt werden. Die **Inhalte** der **Strings** sollen alle **unverändert** bleiben um **folgenden Output** zu erreichen:

```
Day 1 - String Manipulation
String Concatenation is done with the "+" sign.
e.g. print("Hello " + "world")
New lines can be created with a backslash and n.
```

```
print("Day 1 - String Manipulation")
print('String Concatenation is done with the "+" sign.')
print('e.g. print("Hello "+" world")')
print("New lines can be created with a backslash and n(\\n).")
```

Day 1 - String Manipulation
String Concatenation is done with the "+" sign.
e.g. print("Hello "+" world")
New lines can be created with a backslash and n(\\n).

Lösung

Challenge 4

Nutze **print()** und **Escaping**, um den **Output** des Codes **genau so** darzustellen, **wie hier**:

```
"I'm"\
""learning""\
""Python""

print('I\'m\\')
print('""learning""\\')
print('""Python""')
```

```
"I'm"\
""learning""\
""Python""
```

Lösung

```
print('I\'m\\', '""learning""\\', '""Python""', sep='\\n')

"I'm"\
""learning""\
""Python""
```

Playground

```
print("Ich bin ein String.", "Du kannst mich verändern und Verschiedenes  
ausprobieren!")
```

Ich bin ein String. Du kannst mich verändern und Verschiedenes ausprobieren!

Hier hast du Platz, dich auszutoben

Welt 1: Gestrandet

Kapitel D: Variablen

Variablen Basics

Variablen speichern **Inhalt** unter einem **Namen** ab.

Damit kann dann **genau so** gearbeitet werden, wie du es mit ihrem **gespeicherten Inhalt** machen würdest.

```
text = "Ich bin Text"
print(text)
number = 5
print(number)
number_but_text = "5"
print(number_but_text)
print(text, number, number_but_text)
```

```
Ich bin Text
5
5
Ich bin Text 5 5
```

Challenge 5

Benutze **Variablen** und **print()** um den **folgenden Text** auszugeben:

```
Ich bin eine Variable
und ich bin auch dabei.
```

```
Ich bin eine Variable
und ich bin auch dabei.
```

Lösung

```
txt="Ich bin eine Variable\nund ich bin auch dabei."
print(txt)
```

```
Ich bin eine Variable
und ich bin auch dabei.
```

Variablen Namen geben

Es gibt ein paar **Richtlinien** zur **Benennung** von **Variablen**.

1. Reservierte Wörter & feste Regeln:

- Eine Variable kann **nicht** wie ein bestehendes **reserviertes Wort (Keyword)** in Python heißen (z.B. "if")
- Eine Variable kann **nicht** mit einer **Zahl** beginnenn.
- Eine Variable kann **kein Leerzeichen** enthalten.

- Eine Variable kann **bestimmte Zeichen nicht** enthalten (z.B. "-").

1. Richtlinien zur Lesbarkeit & Verständlichkeit:

- Variablen sollten generell so benannt sein, dass aus ihrem Namen ihr **Zweck ersichtlich** ist.
- Bei Variablen mit **nur einem Zeichen** als Namen, sollte man auf die **folgenden Namen verzichten**: "I", "l", "O".
- Man sollte sich für eine **Namensgebungs-Konvention** entscheiden und diese in einem Projekt **beibehalten**.

"Naming Things in Code"

Challenge 6

Deklariere für die folgenden Aussagen eine **präzise, korrekte Variable**.

1. Der Umfang der Schüssel ist 24cm.
2. Der Reifendruck soll 2.5 Bar sein.
3. Der Name der Testperson ist Gustave.
4. Der Geburtstag von Aline ist am 23.05.1995.

Schreibe die Variablen hier

Lösung

```
perimeter="1. Der Umfang der Schüssel ist 24cm."  
pressure="2. Der Reifendruck soll 2.5 Bar sein."  
name="3. Der Name der Testperson ist Gustave."  
birthday="4. Der Geburtstag von Aline ist am 23.05.1995."  
print(perimeter,pressure,name,birthday,sep="\n")
```

1. Der Umfang der Schüssel ist 24cm.
2. Der Reifendruck soll 2.5 Bar sein.
3. Der Name der Testperson ist Gustave.
4. Der Geburtstag von Aline ist am 23.05.1995.

```
print(f"Umfang {perimeter:>90}")
```

Umfang
der Schüssel ist 24cm.

1. Der Umfang

Namenskonventionen

Die drei **geläufigsten Konventionen** für Variablennamen sind: | **Konvention** | **Beispiel** | |
----- | ---- | | **Camel Case** | *eineVariable* | | **Pascal Case** | *EineVariable* | | **Snake Case** |
eine_variable |

```
ichBinCamelCase="Camel Case"  
IchBinPascalCase="Pascal Case"  
ich_bin_snake_case="Snake Case"
```

Playground

```
beispiel_variable = "Ich speichere einen String."  
die_andere_variable = 420  
true_or_false = True  
ich_habe_ein_komma = 32.32
```

Hier hast du Platz, dich auszutoben

Welt 1: Gestrandet

Kapitel E: Datentypen

Datentypen Basics

Datentypen beschreiben, **was für einen Inhalt** wir vor uns haben, z.B. in einer Variable. Man kann sie in Python mit *type()* abfragen.

Typ	Name	Beschreibung
str	String	Text , immer in Anführungszeichen ("" oder "). Kann ALLE Charaktere enthalten.
int	Integer	ganze Zahlen .
float	Float	Zahlen mit Komma .
bool	Boolean	Wahrheitswert . Ist entweder <i>True</i> oder <i>False</i> .

Strings, Integers, Floats & Booleans sind immutable (unveränderbar).

```
text="Was bin ich?"  
print(text)  
print(type(text))
```

```
print(type(25))  
print(type(1.5))  
print(type(True))  
print(type("a"))
```

```
Was bin ich?  
<class 'str'>  
<class 'int'>  
<class 'float'>  
<class 'bool'>  
<class 'str'>
```

Challenge 7

Bei den folgenden Variablen, **überlege zuerst** (und notiere) welchen **Datentyp** jede Variable hat.

Teste dann mit Python, ob deine Überlegungen stimmen.

Hier steht der nötige Basiscode für die Aufgabe

```
name = "Brutus" #str
street = "Zellenstrasse" #str
house_number = 25 #int
height = 1.86 #float
is_married = True #bool
married_date = "15.04.2009" #str
has_kids = False #bool
```

Lösung

Datentypen konvertieren

Inhalte können mit den entsprechenden **Funktionen** in einen bestimmten **Datentyp umgewandelt** werden.

Bei manchen Zieltypen muss der **zu konvertierende Inhalt** eine **bestimmte Form** haben bzw. darf nur bestimmte Charaktere enthalten.

Funktion	Beschreibung	Restriktion
str(X)	konvertiert X in einen String .	Funktioniert bei allen Inhalten .
int(X)	konvertiert X in einen Integer .	Funktioniert nur bei Zahlen .
float(X)	konvertiert X in einen Float .	Funktioniert nur bei Zahlen .
bool(X)	konvertiert X in ein Boolean .	Funktioniert bei allen Inhalten .

Bei einer Konvertierung zu **Boolean** wird immer **True** geliefert, wenn der konvertierte Inhalt nicht leer ist.

Challenge 8

Konvertiere die Variablen so, dass der **vorgegeben Code fehlerfrei läuft** und eine Ausgabe macht.

```
variable1 = "25"
variable2 = 10
variable3 = "Ich bin eine Variable."

print(variable1 + 5) # Die Ausgabe soll "30" sein
print(variable2 + " ist eine Zahl.") # Die Ausgabe soll "10 ist eine Zahl."
                                sein
print(variable3 + 1) # Die Ausgabe soll "2" sein
```

Lösung

```
variable1 = "25"
variable2 = 10
variable3 = "Ich bin eine Variable."

print(int(variable1) + 5) # Die Ausgabe soll "30" sein
print(str(variable2), "ist eine Zahl.") # Die Ausgabe soll "10 ist eine Zahl."
                                     sein
print(bool(variable3) + 1) # Die Ausgabe soll "2" sein

30
10 ist eine Zahl.
2
```

Playground

```
# Integer: Ganzzahl
print(42)
x = 11
print(x) # 11
print(type(x)) # <class 'int'>

# Float: Kommazahl
print(6.789)
print(type(123.45)) # <class 'float'>
# Geht auch ohne führende und endende Null
print(.46) # 0.46
print(2.) # 2.0

# Boolean
# True/False muss genau so geschrieben werden, case-sensitive
print(True) # True
print(False) # False

# Hier hast du Platz, dich auszutoben
```

Welt 1: Gestrandet

Kapitel F: Mathematische Operatoren

Mathematische Operatoren Basics

Python kann **sehr gut rechnen**.

Wir können **mathematische Operatoren** genau so verwenden, **wie wir sie schreiben** würden.

Es gelten grundlegend die gleichen **Hierarchieregeln wie in genereller Mathematik**.

Operation	Symbol	Code
Addition	Plus-Zeichen	+
Subtraktion	Minus-Zeichen	-
Multiplikation	Sternchen	*
Division	Slash/Schrägstrich	/

```
print(1+1)
print(1-1)
print(2*3)
print(2**6)
print(9/2)
print(9//2)
print(9%2)
print(2**3*4-4)
print(10/7)
print(0.1+0.2)
print(3*0.1-0.3)
```

```
2
0
6
64
4.5
4
1
28
1.4285714285714286
0.30000000000000004
5.551115123125783e-17
```

Floats und Rundungsfehler:

<https://www.youtube.com/watch?v=O6FsyQZzJJ0>

IEE 754

Challenge 8

Berechne die folgenden **mathematischen Rechnungen** mit Hilfe von Python Code.

1. $5 \cdot 44 - 5$
2. $42 \div 332 \cdot 42.5$
3. $19 \div 4.25 - 124 \cdot 32$
4. $(42 \div 32) \cdot 4.525$

Hier arbeitest du dir deine Lösung aus.

Lösung

```
print(5 * 44 - 5)
print(2.5 * 44 - 5)
print(19/4.25-124*32)
print(42/32*4.525)
```

```
215
105.0
-3963.529411764706
5.9390625
```

Erweiterte Operatoren

Einige **weitere Operatoren** ermöglichen **zusätzliche mathematische Vorgänge**:

Operation	Symbol	Code
Exponentierung	zwei Sternchen	**
Ganz-Zahl-Division	zwei Slashes/Schrägstriche	//
Restwert-Division	Prozent-Zeichen	%

```
print(2**3)  #23
print(10//5)
print(10//3)
print(10//6)
print(round(10/6))
print(13%5)  #=2Rest3, also 3
```

```
8
2
3
1
2
```

Challenge 9

Löse die folgenden **mathematischen Probleme** mit Hilfe von Python Code.

1. Berechne den Rest der Division von 233 durch 93.
2. Wie oft passt 7 komplett in 325?
3. Der Exponent von 2 ist 11 und dieser Exponent wird ohne Rest durch 3 geteilt.
4. Wir multiplizieren erst alle Zahlen von 1 bis 8 (einschließlich), exponentzieren das Ergebnis mit 3 und berechnen dann den Rest der Divion durch 7.

Hier arbeitest du dir deine Lösung aus.

Lösung

```
print(233%93)
print(325//7)
print(2**((11//3))
print(((1*2*3*4*5*6*7*8)**3%7)
```

47

46

8

0

Playground

```
print(7 + 8)
print(10 - 3)
print(4 * 3)
# Division liefert IMMER einen Float
print(9 / 3)    # 3.0
print(10 / 3)   # 3.3333333333333335
```

```
print(9 // 3)   # 3
print(11 // 3)  # 3
# Wenn mindestens ein Operand ein Float ist, ist auch das Ergebnis ein Float
print(11.0 // 3)    # 3.0
print(11 // 3.0)    # 3.0
print(11.0 // 3.0)  # 3.0
```

```
print(10 % 3)   # 1
# Wenn beide Zahlen gleich sind, bleibt immer 0 übrig:
print(7 % 7)    # 0
# Wenn die erste Zahl kleiner ist, bleibt immer die erste Zahl übrig.
print(4 % 7)    # 4
```

```
# Potenz-Operator: **
# Hoch rechnen
print(2 ** 3)   # 8
print(2 ** 8)   # 256 (8-Bit, 1 Byte)
print(2 ** 16)  # 65K (Short in Java)
print(2 ** 24)  # 16,7M (Anzahl der möglichen Farben)
print(2 ** 32)  # 4,2B (IPv4-Adressen)
```

Hier hast du Platz, dich auszutoben

Welt 1: Gestrandet

Kapitel G: String Manipulation

"Rechnen" mit Strings

Mit **Strings** kann man auch **begrenzt rechnen**.

Hier wird der **Inhalt** des **Strings** entsprechend dem **verwendeten Operator manipuliert**.

Addieren klappt nur **mit anderen Strings** (Datentyp `str/string`).

```
my_string="Ich mag Mathe"
your_string="so überhauprnicht"
print(my_string*2)
print(my_string+your_string)
print(my_string+' '+your_string)
```

```
Ich mag MatheIch mag Mathe
Ich mag Matheso überhauprnicht
Ich mag Mathe so überhauprnicht
```

```
calc_string="5+3"
print(calc_string)
print(eval(calc_string))
```

```
5+3
8
```

eval() berechnet den **Inhalt** eines Strings, als wäre es eine **mathematische Gleichung**. Das klappt **nur**, wenn der **Inhalt ausschließlich** zulässige **mathematische Operationen** enthält.

Challenge 10

Verdopple eine **Seitenlänge** des Rechtecks mit möglichst **kompaktem Code**.

```
print("#####")
print("#      #")
print("#      #")
print("#####")
```

```
#####
#      #
#      #
#####
```

Lösung

```
print("#####"*2)
print("#"+' '*2+"#")
print("#"+' '*2+"#")
```

```

print("#####"*2)

for i in range(5):
    if i%3==2:
        continue
    if i%2==0:
        print("#####"*2)
    else:
        print("#" + '          '*2 + "#")

```

```

#####
#          #
#          #
#####
#####
#          #
#          #
#####

```

String Index

Mit Angaben in **eckigen Klammern** `//` können wir **mit Strings interagieren**.
Jedes Zeichen in einem String ist ein **Index** (auch Leerzeichen, Satzzeichen, etc.).
Die **Rückgabe** ist das **Zeichen**, was an der **Stelle des angegebenen Index** steht.

STRING[INDEX]

```

my_string2="String"
print(my_string2[0])
print(my_string2[3])
print(my_string2[5])

print(my_string2[0]+my_string2[3])

```

```

S
i
g
Si

```

Challenge 11

Nutze String Indexe, um aus dem vorgegeben String **bestimmte Ausgaben** zu erzeugen.

1. "n"
2. "ein"
3. "Sieben"

```

my_string = "Ich bin ein String."
#-----012345678901234567

```

Lösung

```
print(my_string[6])
print(my_string[8:11])
print(my_string[12]+my_string[5]+my_string[8]+my_string[4]+my_string[8]+my_string[6])

n
ein
Sieben
```

String Slicing

String Slicing basiert auf **Indexen**.

Statt nur einen Index auszugeben, können wir uns **beliebig viele** entsprechend der **angegebenen Regeln** ausgeben lassen.

STRING[ANFANG:ENDE:SCHRITTWEITE]

Defaults: *STRING[0:ENDE:1]*

Die Angaben sind **immer 0 basiert**, also das **erste Zeichen** hat den **Index = 0**.

Wenn wir ein **Ende** angeben, ist dies immer **exklusiv**.

Zum Beispiel: *[:8]* bedeutet alles **von Index 0 bis einschließlich 7**.

```
def frog_jump(distance):
    first_jump = 1.0
    last_jump = first_jump
    rest_distance = distance - first_jump
    jumpers = 1

    while rest_distance < distance:
        if jumpers == 20:
            print(f"Der Frosch schafft es nicht, mit {jumpers} Sprüngen die
                Straße zu überqueren.")
            print(f"Gesprungene Strecke: {distance - rest_distance}m")
            return
        next_jump = last_jump / 2
        rest_distance -= next_jump
        last_jump = next_jump
        jumpers += 1
        print(jumpers)

    print(f"Der Frosch braucht {jumpers} Sprünge, um die Straße zu
        überqueren.")
```

```
frog_jump(2.5)
```

```
2
3
4
5
```

6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

Der Frosch schafft es nicht, mit 20 Sprüngen die Straße zu überqueren.
Gesprungene Strecke: 1.9999980926513672m

Man kann auch **negative Angaben** verwenden.
Dann laufen wir den **String von rechts** durch.

```
my_string = "String"
print(my_string[2:3])
print(my_string[0:6:2])
print(my_string[::-2])
print(my_string[4:8:2])
print(my_string[0:11:1])
```

```
r
Srnr
Srnr
n
String
```

```
my_string="String"
print(my_string[::-1])
print(my_string[-4:])
```

```
gnirts
ring
```

```
#index:      0123456
#           //
my_string = ["abcdefg"]

print(my_string[5:2:1])

[]
```

Challenge 12

Nutze **String Slicing**, um aus dem vorgegeben String die **folgenden Ausgaben** zu erzeugen.

Versuche, deinen Code jeweils **so kompakt wie möglich** zu machen.

1. b nueTtad sleV rgünaS
2. tsnnak nebü sgnirtS nov necilS sad ud med na ,txeT rezruk nie nib hcI
3. Iinz
4. Ich bin ein String.

```
my_string = "Ich bin ein kurzer Text, an dem du das Slicen von Strings üben  
kannst."
```

Lösung

```
my_string = "Ich bin ein kurzer Text, an dem du das Slicen von Strings üben  
kannst."  
#  
012345678901234567890123456789012345678901234567890123456789
```

```
print(my_string[4::3])  
print(my_string[68::-1])  
print(my_string[0:20:5])  
print(my_string[0:11]+my_string[49:56]+my_string[-1])
```

```
b nueTtad  sleV rgünaS  
tsnnak nebü sgnirtS nov necilS sad ud med na ,txeT rezruk nie nib hcI  
Iinz  
Ich bin ein String.
```

```
def dezimal_zu_basis_n(zahl, basis):  
    zeichen = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"  
    if zahl == 0:  
        return '0'  
    basis_zahl = ''  
    while zahl > 0:  
        rest = zahl % basis  
        basis_zahl = zeichen[rest] + basis_zahl  
        zahl = zahl // basis  
    return basis_zahl
```

```
b = int(input("Geben Sie die Basis (2-36) ein: "))  
while b < 2 or b > 36:  
    b = int(input("Ungültige Basis! Bitte geben Sie eine Basis zwischen 2 und  
36 ein: "))
```

```
dezimalzahl = int(input("Geben Sie eine Dezimalzahl ein: "))  
print(f"Die Dezimalzahl {dezimalzahl} zur Basis {b} ist:  
{dezimal_zu_basis_n(dezimalzahl, b)}")
```

Die Dezimalzahl 4095 zur Basis 16 ist: FFF

Playground

```
s = 'Hello World'
print(s[0]) # H
print(s[6]) # W

# Slicing per Teilbereichsoperator
# ANFANG ist inklusive und ENDE ist exklusive.
s = 'Hello World'
print(s[1:3]) # eL
print(s[0:3]) # Hel
# 0 als Anfang ist Standard und kann weggelassen werden:
print(s[:3]) # Hel

# Wird die zweite Zahl weggelassen, wird der String bis zum Ende
# eingeschlossen:
print(s[1:]) # ello World

# Negative Indexe
s = 'Hello World'
print(s[-1]) # d
print(s[-3:-1]) # rL
# Vom x-Zeichen bis zum Ende
print(s[-3:]) # rld

# Schrittweite
# [ANFANG:ENDE:SCHRITTWEITE]
s = 'Hello World'
print(s[1:10:3]) # eoo
print(s[::2]) # HLoWrD
print(s[1::3]) # eood

# Man darf einen Index benutzen, den es nicht gibt
print(s[1:11:3]) # eood
print(s[1:1000:3]) # eood

# Negative Schrittweite mit 1 als Standard
# Fängt von hinten an
# ANFANG muss größer sein als ENDE.
print(s[10:2:-2]) # drWo
# Den kompletten String umdrehen:
print(s[::-1]) # dlroW olleH

# ENDE muss immer größer (im Sinne von weiter rechts) sein als ANFANG,
# sonst gibt es keine Fehlermeldung, sondern einen leeren String.
print('Keine Ausgabe', s[4:0:1]) # Keine Ausgabe
# Bei negativer Schrittweite genau umgekehrt:
print('Keine Ausgabe', s[0:4:-1]) # Keine Ausgabe

s = 'Hello World'
print('Keine Ausgabe', s[-1:-4]) # Keine Ausgabe
print(s[-4:-1]) # orl
```

```

# String-Operatoren: +, *
# + Verkettung, Konkatenation (concatenation)
x = 'Hello'
y = 'World'
z = x + ' ' + y
print(z) # Hello World

# + Multiplikation mit Integer
print('Hello' * 10) # HelloHelloHelloHelloHelloHelloHelloHelloHelloHello

s = 'Hello World'
print(s[1:11:3]) # eood

# String per dreifacher Anführungszeichen definieren
str1 = '''Test'''
print(str1) # Test

str2 = '''Test2
Test2'''
print(str2)
# Test2
# Test2

# Hier hast du Platz, dich auszutoben

```

Welt 1: Gestrandet

Kapitel H: f-Strings

f-Strings

Mit **f-Strings** können wir uns **innerhalb eines Strings** elegant **Variablen** ausgeben lassen.

So können wir in einem **einzelnen String** **beliebig viele Variablen** ausgeben.

f-Strings helfen einen **Output im Code übersichtlicher** zu machen.

Man schreibt sie **innerhalb eines Strings** mit **geschweiften Klammern { }**.

```

distance=250
print(f"Die Strecke ist {distance}km lang.")

```

Die Strecke ist 250km lang).

```
Schuessel = 24
Reifendruck = 2.5
Testperson = "Gustave"
AGeb = "23.05.1995"
```

```
print(f"Der Umfang der Schüssel ist", Schuessel, ".", sep="")
print(f"Der Reifendruck soll", Reifendruck, "Bar sein.")
print(f"Der RName der Tespersion ist", Testperson, ".", sep="")
print(f"Der Geburtstag von Aline ist am ", AGeb, ".", sep="")
#Das geht auch ohne f-String
```

```
Der Umfang der Schüssel ist24.
Der Reifendruck soll 2.5 Bar sein.
Der RName der Tespersion istGustave.
Der Geburtstag von Aline ist am 23.05.1995.
```

```
drive_time=45
break_time=5
print(f"Die Fahrt dauert mit Pausen {drive_time+break_time} Minuten.")

print(f"Die Fahrt dauert mit Pausen und Puffer {drive_time+break_time+5}
      Minuten.")
```

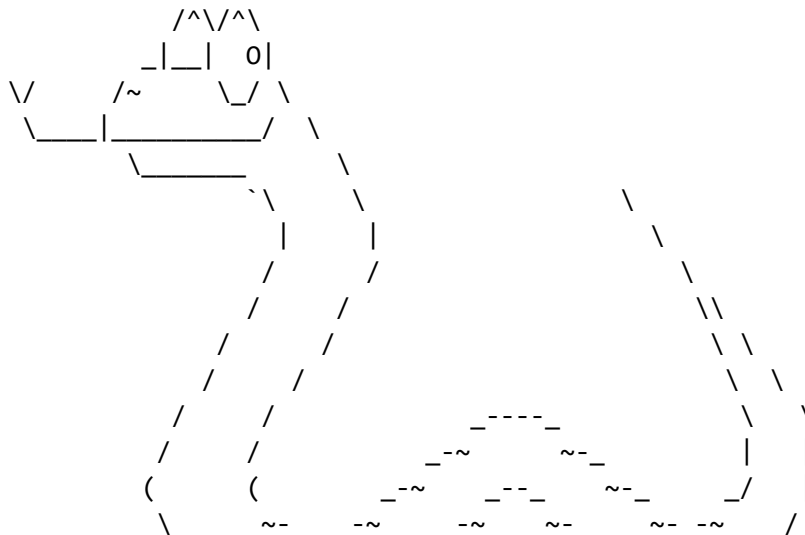
```
Die Fahrt dauert mit Pausen 50 Minuten.
Die Fahrt dauert mit Pausen und Puffer 55 Minuten.
```

Wir können auch **innerhalb des f-Strings** mit den Variablen interagieren, z.B. **rechnen**.

Challenge 13

Löse die folgende Problemstellung mit Hilfe von Python und verpacke sie **in einem f-String**.

Der Preis für eine Kokosnuss auf Pysland ist aktuell 2.5 Münzen, mit einem Steuersatz von 11%.
 Py möchte für ihre Party 18 Kokosnüsse kaufen.
 Gib Py eine gut lesbare Ausgabe, wie viel sie für ihren Einkauf zahlen muss.



~ _ ~ _ ~
~ _ ~ _ ~

```
num_coconut = 18  
price_coconut = 2.5
```

Lösung

```
num_coconut = 18  
price_coconut = 2.5  
steuersatz = 0.11  
print(f"Der Gesamtpreis für die {num_coconut} Kokosnüsse a {price_coconut}  
Münzen inklusive {100*steuersatz} % Steuern beträgt:  
{num_coconut*price_coconut*(1+steuersatz)} Münzen")
```

Der Gesamtpreis für die 18 Kokosnüsse a 2.5 Münzen inklusive 11.0 % Steuern beträgt: 49.95 Münzen

Playground

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel A: Ausgaben formatieren

Zahlen formatieren

Du kannst **Zahlen in der Ausgabe, vorallem bei f-Strings**, formatieren.
Dies ist **besonders** beim **Ausgeben** von **bestimmten Einheiten sinnvoll** (z.B. Euro immer mit 2 Stellen: 12.00€).
Es kann Ausgaben auch **einheitlicher, übersichtlicher** und **schöner** aussehen lassen.

Bezeichnung	Syntax	Beschreibung
Nachkommastellen	<code>:.Xf</code>	wobei X die Anzahl der Nachkommastellen angibt.
Tausender-Separator	<code>:.X</code>	wobei X der Separator , der zwischen allen Tausender-Schritten eingefügt wird.
Totale Stellen	<code>:Y>X</code>	wobei X die Anzahl der gesamten Stellen angibt, bei mehr als vorhanden wird links mit Y gefüllt.

Bezeichnung	Syntax	Beschreibung
Totale Stellen	:Y<X	wobei X die Anzahl der gesamten Stellen angibt, bei mehr als vorhanden wird rechts mit Y gefüllt.

```

num1=1.5
num2=2.5
num3=2146545
print(f"Keine Nachkommastelle: {num1:.0f}") #ungerade Zahlen werden
    aufgerundet (bei .5)!
print(f"Keine Nachkommastelle: {num2:.0f}") #gerade Zahlen werden abgerundet
    (bei .5)!
print(f"Mit 2 Kommastellen: {num3:.2f}")
print(f"Tausenderseperator: {num3:,}")
print(f"Tausenderseperator: {num3:_}")
print(f"Fünf totaleStellen (rechtsbündig): {num1:0>5}") #mit 0 auffüllen
print(f"Fünf totaleStellen (rechtsbündig): {num1:_>5}") #mit _ auffüllen
print(f"Fünf totaleStellen (linksbündig): {num1:_<5}")
print(f"Fünf totaleStellen (rechtsbündig): {num1:>5}") #ohne auffüllen
print(f"Fünf totaleStellen (mittig): {num1:^5}")

```

```

Keine Nachkommastelle: 2
Keine Nachkommastelle: 2
Mit 2 Kommastellen: 2146545.00
Tausenderseperator: 2,146,545
Tausenderseperator: 2_146_545
Fünf totaleStellen (rechtsbündig): 001.5
Fünf totaleStellen (rechtsbündig): __1.5
Fünf totaleStellen (linksbündig): 1.5__
Fünf totaleStellen (rechtsbündig):   1.5
Fünf totaleStellen (mittig):  1.5
2.0

```

Challenge 14

Schreibe eine Ausgabe für die jeweiligen Aussagen, die für die entsprechende **Einheit**, den **Kontext** und **gute Lesbarkeit** formatiert sind.

1. Insgesamt ist die Strecke 2,252km lang.
2. Das Produkt kostet 2.20€.
3. Der Gesamtpreis beträgt 1,454.00€.

```

distance = 2252
price_product = 2.2
price_total = 1454

```

Lösung

```

distance = 2252
price_product = 2.2

```

```
price_total = 1454
print(f"Insgesamt ist die Strcke {distance:,}km lang.")
print(f"Das Produkt kostet {price_product:.2f}€.")
print(f"Der Gesamtpreis beträgt {price_total:,.2f}€.")
```

Insesamt ist die Strcke 2,252km lang.
Das Produkt kostet 2.20€.
Der Gesamtpreis beträgt 1,454.00€

Playground

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel B: Listen

Listen Basics

Mit **Listen** können **mehrere Elemente** in ein **Objekt** gefasst werden.
Es können **beliebig Datentypen kombiniert** werden.
Strings können in **Listen** überführt werden, wobei **jeder Index ein eigenes Listenelement** wird (auch Leerzeichen).

Listen sind mutable (veränderbar).

```
my_list=[1,"String" ,3,4.5,5,True]
print(my_list)
print(my_list[3])
print(type(my_list))
```

```
[1, 'String', 3, 4.5, 5, True]
4.5
<class 'list'>
```

```
my_string="Hello World"
my_list2=list(my_string)
print(my_list2)
print(*my_list2)
print(*my_list2,sep='')
```

```
['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd']
H e l l o   W o r l d
Hello World
```

```

my_list3=[1,2,3,4,5]
print(my_list3[3])
my_list3[3]=0
print(my_list3[3])
print(my_list3)

4
0
[1, 2, 3, 0, 5]

zahlen = [1, 2, 3, 4]
namen = ["Anna", "Ben", "Chris"]
gemischt = [1, "Hallo", 3.5, True]

print("Standart Zahlenliste")
print(zahlen)
zahlen.append(5)      # an Letzter Position eine 5 einfügen
print("Liste nach dem einfügen einer Zahl an letzter Stelle")
print(zahlen)
zahlen.insert(1, 99)  # an Position 1 eine 99 einfügen
print("einfügen der Zahl 99 an 2ter Stelle in der Liste Index[1]")
print(zahlen)

print("Standart Namenliste")
print(namen)
print("Name Hans an lezter stelle der Liste einfügen ")
namen.append("Hans")    # an Letzter Position Hans einfügen
print(namen)
print("Name Wurst an 3 stelle der Liste = Index[2] einfügen ")
namen.insert(2, "Wurst") # an Position 3 Wurst einfügen
print(namen)

print(gemischt)

```

```

Standart Zahlenliste
[1, 2, 3, 4]
Liste nach dem einfügen einer Zahl an letzter Stelle
[1, 2, 3, 4, 5]
einfügen der Zahl 99 an 2ter Stelle in der Liste Index[1]
[1, 99, 2, 3, 4, 5]
Standart Namenliste
['Anna', 'Ben', 'Chris']
Name Hans an lezter stelle der Liste einfügen
['Anna', 'Ben', 'Chris', 'Hans']
Name Wurst an 3 stelle der Liste = Index[2] einfügen
['Anna', 'Ben', 'Wurst', 'Chris', 'Hans']
[1, 'Hallo', 3.5, True]

```

Challenge 15

Nutze Listen, um die folgenden Probleme zu **lösen**:

1. Wandle den gegebenen String in eine Liste um und gebe diese aus.
2. Gebe den Inhalt der Liste zusammenhängend aus (so, wie der String am Anfang).

```
my_string = "I'm a string!"
```

Lösung

```
my_string = "I'm a string!"
my_list4=list(my_string)
print(my_list4)
print(*my_list4,sep='')
```

```
['I', "'", 'm', ' ', 'a', ' ', 's', 't', 'r', 'i', 'n', 'g', '!']
I'm a string!
```

Listen Slicen

Wir können **Listen** genau **wie Strings slicen** .
Hierbei wird **jeweils** das **Element** im jeweiligen **Listen-Index** ausgegeben.

Challenge 16

Nutze **List-Slicing** , um mit der gegebenen Liste folgende **Outputs** zu erreichen.

1. hi world
2. wild
3. list

```
my_list = ["d", "l", "r", "o", "w", " ", "i", "h", "s", "t"]
#          0   1   2   3   4   5   6   7   8   9
```

Lösung

```
print(*my_list[7::-1],sep='')
print(my_list[4]+my_list[6]+my_list[1]+my_list[0])
print(my_list[1]+my_list[6]+my_list[8]+my_list[9])
```

```
hi world
wild
list
```

Listen Verschachteln

Listen können auch **weitere Listen** **beinhalten**.
Dabei sprechen wir mit dem **Index** die **gesamte Liste** an, die an diesem Index **in unserer Liste** liegt.

```

my_list=[1,2,3,[4,5]]
print(my_list[3])
print(my_list)
print(my_list[3][0])

my_list_nested=[1,[2,[3,[4,5]]]]
print(my_list_nested)
print(my_list_nested[1][1][1][0])

[4, 5]
[1, 2, 3, [4, 5]]
4
[1, [2, [3, [4, 5]]]]
4

```

Challenge 17

Bilde die folgenden Sätze, in dem du **Daten** aus den **gegebenen Listen** extrahierst:

1. hi I am home
2. thanks for the help
3. yes go home

```

list1 = [["hi", "two"], "yes", "house",[["home", "go"], "leave"]]
list2 = ["thanks", "help", ["for", ["the", "great"], "good"], "am", "I"]

```

Lösung

```

list1 = [["hi", "two"], "yes", "house",[["home", "go"], "leave"]]
list2 = ["thanks", "help", ["for", ["the", "great"], "good"], "am", "I"]
print(list1[0][0],list2[-1],list2[-2],list1[3][0][0])
print(list2[0],list2[2][0],list2[2][1][0],list2[1])
print(list1[1],list1[3][0][1],list1[3][0][0])
print(list2[2][1][1])

```

```

hi I am home
thanks for the help
yes go home
great

```

Playground

```

liste1 = [1, 3, 5, 7]
print(liste1) # [1, 3, 5, 7]
print(type(liste1)) # <class 'list'>

s = 'Hello'
liste2 = list(s)
print(liste2) # ['H', 'e', 'l', 'l', 'o']

```

```

# Listen können alle Datentypen (auch gemischt) beinhalten:
liste3 = [True, 23, 9.4, 'Hello', [7, 11]]
print('liste:', liste3) # liste: [True, 23, 9.4, 'Hello', [7, 11]]

# Mehrdimensionale Listen
liste4 = [1, 2, 3, [4, 5]]
print(liste4) # [1, 2, 3, [4, 5]]
print(liste4[3]) # [4, 5]
print(liste4[3][0]) # 4
print(liste4[3][1]) # 5

liste5 = [1, 2, [3, 4, 5, [6, 7, 8, 9]]]
print(liste5) # [1, 2, [3, 4, 5, [6, 7, 8, 9]]]
print(liste5[2][3][2]) # 8
print([1, 2, [3, 4, 5, [6, 7, 8, 9]]][2][3][2]) # 8
print([3, 4, 5, [6, 7, 8, 9]][3][2]) # 8
print([6, 7, 8, 9][2]) # 8

liste5 = [
    1,
    2,
    [
        3,
        4,
        5,
        [
            6,
            7,
            8,
            9
        ]
    ]
]

# In eine sliced Liste zurückschreiben:
x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
x[::2] = (10, 20, 30, 40, 50)
# x[::2] = 10, 20, 30, 40, 50, 60 # ValueError
print(x) # [10, 2, 20, 4, 30, 6, 40, 8, 50]

x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
y = (10, 20, 30, 40, 50)
c = 0
for i in y:
    x[c] = i
    c += 2
print(x)

# Hier hast du Platz, dich auszutoben

```

Welt 2: Wald

Kapitel C: Listen Manipulation

Mit Listen interagieren

Es gibt **viele Methoden**, um **mit Listen** zu interagieren.

Methoden werden immer **auf ihr Objekt angewendet**, also:

LIST.methode(PARAMETER)

Methode	Beschreibung
<i>append(OBJECT)</i>	fügt das übergebene OBJECT am Ende hinzu .
<i>insert(INDEX, OBJECT)</i>	fügt das übergebene OBJECT am angegebenen INDEX hinzu.
<i>extend(OBJECT)</i>	erweitert die Liste um die Indexe des übergebenen, iterable OBJECT .
<i>count(OBJECT)</i>	gibt die Anzahl der Vorkommnisse des übergebenen OBJECT zurück.
<i>index(OBJECT)</i>	gibt den Index des ersten Vorkommnisses des übergebenen OBJECT zurück.
<i>pop(INDEX)</i>	entfernt das OBJECT mit dem angegebenen INDEX und gibt dieses zurück . Ohne Index wird das letzte entfernt.
<i>remove(OBJECT)</i>	entfernt das erste Vorkommnis des übergebenen OBJECT .
<i>reverse()</i>	kehrt die Liste um .
<i>sort()</i>	sortiert die Liste aufsteigend .
<i>clear()</i>	löscht den Inhalt der Liste.

```
my_list=[1,2,3,4]
print(my_list)
#append
my_list.append(5)
print(my_list)
#insert
my_list.insert(0,-1)
my_list.insert(-1,0)
print(my_list)
#extend
my_list.extend([5,6,7,8,9])
print(my_list)
#count
print(my_list.count(5))
#index
```

```

print(my_list.index(0))
#pop
my_list.pop(7)
print(my_list.pop(0))
print(my_list)
#remove
my_list.extend([3,3,3])
my_list.remove(3) #das erste vorkommende wird gelöscht
print(my_list)
#reverse
print(my_list[::-1])
my_list.reverse() #Liste wird verändert
print(my_list)
#sort
my_list.sort()
print(my_list)
#rückwärts
my_list.sort(reverse=True)
print(my_list)

```

```

my_list2=[5,7,8,9,-2,'o',True]
my_list+=my_list2
print(my_list)

```

```

#clear
my_list.clear()
print(my_list)

```

```

[1, 2, 3, 4]
[1, 2, 3, 4, 5]
[-1, 1, 2, 3, 4, 0, 5]
[-1, 1, 2, 3, 4, 0, 5, 5, 6, 7, 8, 9]
2
5
-1
[1, 2, 3, 4, 0, 5, 6, 7, 8, 9]
[1, 2, 4, 0, 5, 6, 7, 8, 9, 3, 3, 3]
[3, 3, 3, 9, 8, 7, 6, 5, 0, 4, 2, 1]
[3, 3, 3, 9, 8, 7, 6, 5, 0, 4, 2, 1]
[0, 1, 2, 3, 3, 3, 4, 5, 6, 7, 8, 9]
[9, 8, 7, 6, 5, 4, 3, 3, 3, 2, 1, 0]
[9, 8, 7, 6, 5, 4, 3, 3, 3, 2, 1, 0, 5, 7, 8, 9, -2, 'o', True]
[]

```

```

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]
vals = nums[0:5]
my_var = vals[:]

```

```

print(id(nums))

```

```

print(nums)
print(id(vals))
print(vals)
print(id(my_var))
print(my_var)

nums = [1, 2, 3]
vals = nums
del vals[1:2]

print(id(nums))
print(nums)
print(id(vals))
print(vals)

my_string = list("Hello")
another_string = my_string
my_string = list("Bye")

print(id(my_string))
print(my_string)
print(id(another_string))
print(another_string)

```

Challenge 18

Manipuliere die vorgegebene Liste um die jeweilige neue Liste zu erhalten. NICHT die Liste komplett neu erstellen.

1. [1, 3, "Hi", True]
2. [1, "Hi", True]
3. [1, "Hi", True, 5, 7, 9]
4. [9, 7, 5, True, "Hi", 1]
5. [9, 7, 5, True, "Hi", 1, "Bye"]
6. [1, 5, 7, 9]

```

my_list = [1, 3, "Hi", True, 9]

my_list.pop(-1)
print(my_list)
my_list.remove(3)
print(my_list)
my_list.reverse()
my_list.insert(0,5)
my_list.insert(0,7)
my_list.insert(0,9)
print(my_list)
my_list.append("Bye")
print(my_list)
my_list.remove(True)
my_list.remove("Hi")

```

```

my_list.remove("Bye")
my_list.sort()
print(my_list)

[1, 3, 'Hi', True]
[1, 'Hi', True]
[9, 7, 5, True, 'Hi', 1]
[9, 7, 5, True, 'Hi', 1, 'Bye']
[1, 5, 7, 9]

```

Lösung

Playground

```

# append()
liste1 = [1, 2, 3, 4]
print(liste1) # [1, 2, 3, 4]
liste1.append(5)
print(liste1) # [1, 2, 3, 4, 5]

# insert(index, object)
liste1 = [1, 2, 3, 4]
liste1.insert(0, 9)
print(liste1) # [9, 1, 2, 3, 4]
liste1.insert(2, 11)
print(liste1) # [9, 1, 11, 2, 3, 4]
# Wenn der Index zu groß ist, wird angehängen
liste1.insert(30, 42)
print(liste1) # [17, 1, 11, 2, 3, 4, 42]
# Geht auch mit negativen Indexen

# Zum Überschreiben eines Wertes benutzt man Indexing
liste1 = [1, 2, 3, 4]
liste1[0] = 17
print(liste1) # [17, 2, 3, 4]

# count(object)
print([1, 2, 3, 1, 4, 1, 7, 1].count(1)) # 4

# index(object)
print([1, 2, 3, 4, 2, 7, 2, 1].index(2)) # 1

# pop()
liste1 = [1, 2, 3, 4]
liste1.pop()
print(liste1) # [1, 2, 3]
print(liste1.pop()) # 3
print(liste1) # [1, 2]

# pop(index)

```

```

liste1 = [1, 2, 3, 4]
print(liste1.pop(2)) # 3
print(liste1) # [1, 2, 4]

# remove(value)
liste2 = [1, 2, 3, 4, 2, 7, 2, 1]
liste2.remove(2)
print(liste2) # [1, 3, 4, 2, 7, 2, 1]
print(liste2.remove(2)) # None
print(liste2) # [1, 3, 4, 7, 2, 1]

# reverse()
liste1 = [1, 2, 3, 4]
liste1.reverse()
print(liste1) # [4, 3, 2, 1]

# sort()
liste2 = [1, 2, 3, 4, 2, 7, 2, 1]
liste2.sort()
print(liste2) # [1, 1, 2, 2, 2, 3, 4, 7]

# extend(iterable)
liste1 = [1, 2, 3, 4]
liste1.extend([5, 6, 7])
print(liste1) # [1, 2, 3, 4, 5, 6, 7]
liste1.extend('Hello')
print(liste1) # [1, 2, 3, 4, 5, 6, 7, 'H', 'e', 'l', 'l', 'o']

# clear()
liste1 = [1, 2, 3, 4]
liste1.clear()
print(liste1) # []

# Hier hast du Platz, dich auszutoben

```

Welt 2: Wald

Kapitel D: Length & Range

len()

Mit *len()* überprüft man, **wie lang** ein **Objekt** ist bzw. **wie viele Elemente** es hat.

Dies funktioniert **nur** für **Objekte**, die einen **Index** haben:
Strings, Lists, Dictionaries, Ranges.

```
string="Ich bin ein String."  
print(len(string))  
my_list=[1,2,3,4,5]  
print(len(my_list))
```

19

5

Challenge 19

Finde mit Hilfe von *len()* **Details** zu folgenden Variablen **heraus**:

1. Wie viele Zeichen sind in "greeting"?
2. Wie viele Indexe sind in "my_list"?

```
greeting = "Heya, willkommen in diesem Teil des Python Kurses!"  
my_list = list(range(0, 200, 7))
```

Lösung

```
greeting = "Heya, willkommen in diesem Teil des Python Kurses!"  
my_list = list(range(0, 200, 7))  
print(len(greeting))  
print(len(my_list))
```

50

29

range()

range() ist eine **generative Funktion**, welche ein **range-Objekt** erstellt und entsprechend der Parameter füllt.

Funktioniert **wie Slicing** von Strings oder Listen.

Besonders für das **Erstellen** von **Listen**, **Dictionaries** und in **for-Loops** ist dies nützlich.

```
print(range(10))  
print(type(range(10)))  
print(len(range(10)))  
my_list=list(range(10))  
print(my_list)  
my_list2=list(range(1,11,2))  #11 exklusiv!  
print(my_list2)  
my_list3=list(range(5,16))  
print(my_list3)
```

```
range(0, 10)  
<class 'range'>  
10  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[1, 3, 5, 7, 9]
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

```
my_range=range(13,104,7)
print(len(my_range))
```

```
print(my_range[8])
print(my_range[3:8])
print(*my_range)
```

```
13
69
range(34, 69, 7)
13 20 27 34 41 48 55 62 69 76 83 90 97
```

```
[print(x) for x in range(7,106,7)]
```

```
7
14
21
28
35
42
49
56
63
70
77
84
91
98
105
```

```
[None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None,
None]
```

Challenge 20

Nutze *range()* und *len()* um die folgenden Probleme zu lösen:

1. Erstelle eine Liste mit allen ungeraden Zahlen von 1 bis 100.
2. Wie lang ist diese Liste?
3. Erstelle eine zweite Liste mit jeder durch 7 teilbaren Zahl bis 777.
4. Wie lang ist diese zweite Liste?

Hier kommt deine Lösung hinmy_list

Lösung

```
my_list=list(range(1,100,2))
print(len(my_list))
my_list2=list(range(7,778,7))
print(len(my_list2))
```

```
50
111
```

Playground

```
print(type(range(10))) # <class 'range'>

# range() gibt ein range-Objekt zurück,
# was von list() zu einer Liste gemacht wird.
# Der rechte Rand (das ENDE) ist exklusiv.
print(list(range(10))) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Bei zwei Parameter/Argumenten sind es ANFANG und ENDE.
# ANFANG ist inklusiv und ENDE ist exklusiv
print(list(range(0, 10))) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(range(4, 10))) # [4, 5, 6, 7, 8, 9]

# Schrittweite
print(list(range(1, 10, 2))) # [1, 3, 5, 7, 9]
print(list(range(0, 10, 2))) # [0, 2, 4, 6, 8]

# Wenn ANFANG größer (oder gleich) ENDE ist, entsteht ein Leeres range-
  Objekt:
print(list(range(10, 2))) # []
print(type(range(10, 2))) # <class 'range'>

# Negative Schrittweite
# ANFANG muss größer als ENDE sein:
print(list(range(10, 0, -2))) # [10, 8, 6, 4, 2]
print(list(range(10, 0, -1))) # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
print(list(range(10, -1, -1))) # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

# Indexing funktioniert auch mit dem range-Objekt:
print(range(13, 104, 7)) # range(13, 104, 7)
print(list(range(13, 104, 7))) # [13, 20, 27, 34, 41, 48, 55, 62, 69, 76,
  83, 90, 97]
print(range(13, 104, 7)[6]) # 55
```

```

# Slicing funktioniert auch mit dem range-Objekt:
print(range(13, 104, 7)[4:6])      # range(41, 55, 7)

print(list(range(13, 104, 7)[4:6])) # [41, 48] - Hier wird das range-Objekt
    gesliced
print(list(range(13, 104, 7))[4:6]) # [41, 48] - Hier wird die Liste
    gesliced

# Alle Leeren range-Objekte werden zu False.
# Alle range-Objekte mit irgendeinem Inhalt werden zu True.
print(bool(range(0)))      # False
print(bool(range(7, 1)))   # False
print(list(range(7, 1)))   # []
print(bool(range(7)))      # True

# Zählschleife mit for & range()
# for(i = 0; i < 10; i++)
for i in range(10):
    print(i, end=' ') # 0 1 2 3 4 5 6 7 8 9
print()

# for(i = 3; i < 8; i++)
for i in range(3, 8):
    print(i, end=' ') # 3 4 5 6 7
print()

# for(i = -13; i < 8; i += 4)
for i in range(-13, 8, 4):
    print(i, end=' ') # -13 -9 -5 -1 3 7
print()

# Leeres range-Objekt
for _ in range(0):
    print("NEVER")

# Q309
# data = [[x for x in range(y)] for y in range(3)]
data = [1 for _ in range(3)]
print(data) # [1, 1, 1]

data = [[] for _ in range(3)]
print(data) # [], [], []

# Hier hast du Platz, dich auszutoben

```

Welt 2: Wald

Kapitel E: Vergleichs-Operatoren

Gängige Operatoren

Vergleichs-Operatoren prüfen den **Wahrheitswert** einer Aussage.
Ist die angegebene **Operation wahr**, wird **True** ausgegeben, andernfalls **False**.

Operator	Bedeutung	Beschreibung
<	kleiner	was links steht, hat einen kleineren Wert als rechts .
>	größer	was links steht, hat einen größeren Wert als rechts .
<=	kleiner oder gleich	was links steht, hat einen kleineren oder den gleichen Wert als rechts .
>=	größer oder gleich	was links steht, hat einen größeren oder den gleichen Wert als rechts .
==	genau gleich	was links steht, hat genau den gleichen Wert (bzw. Inhalt) wie rechts.
!=	nicht gleich	was links steht, hat nicht genau den gleichen Wert (bzw. Inhalt) wie rechts.

Challenge 21

Überprüfe die folgenden **Aussagen** mit Hilfe von **Vergleichs-Operatoren**:

1. Terry ist größer als Luna.
2. Maeve ist nicht genau so groß wie Franz.
3. Maeve ist mindestens genau so groß wie Luna und kleiner als Terry.
4. Terry ist größer als Franz und mindestens genau so groß wie Maeve.
5. Franz ist kleiner als Luna, aber größer als Maeve.

```
luna = 163
franz = 174
maeve = 181
terry = 204
```

Lösung

```
luna = 163
franz = 174
maeve = 181
terry = 204
print(terry>luna)
print(maeve!=franz)
print(maeve>=luna and maeve<terry)
print(terry>franz and terry>=maeve)
print(franz<luna and franz>maeve)
```

True
True
True
True
False

String Vergleiche

Beim **Vergleichen von Strings** entscheidet der entsprechende **UTF-8 Code** des **ersten Zeichens**.

Wenn das **erste Zeichen** beider Strings **gleich** ist, geht es zum **zweiten**, etc.

Scan- code	ASCII hex dez	Zeichen	Scan- code	ASCII hex dez	Zch.	Scan- code	ASCII hex dez	Zch.	Scan- code	ASCII hex dez	Zch.
	00 0	NUL ^@		20 32	SP		40 64	@	0D	60 96	`
	01 1	SOH ^A	02	21 33	!	1E	41 65	A	1E	61 97	a
	02 2	STX ^B	03	22 34	"	30	42 66	B	30	62 98	b
	03 3	ETX ^C	29	23 35	#	2E	43 67	C	2E	63 99	c
	04 4	EOT ^D	05	24 36	\$	20	44 68	D	20	64 100	d
	05 5	ENQ ^E	06	25 37	%	12	45 69	E	12	65 101	e
	06 6	ACK ^F	07	26 38	&	21	46 70	F	21	66 102	f
	07 7	BEL ^G	0D	27 39	'	22	47 71	G	22	67 103	g
0E	08 8	BS ^H	09	28 40	(	23	48 72	H	23	68 104	h
0F	09 9	TAB ^I	0A	29 41	)	17	49 73	I	17	69 105	i
	0A 10	LF ^J	1B	2A 42	*	24	4A 74	J	24	6A 106	j
	0B 11	VT ^K	1B	2B 43	+	25	4B 75	K	25	6B 107	k
	0C 12	FF ^L	33	2C 44	,	26	4C 76	L	26	6C 108	l
1C	0D 13	CR ^M	35	2D 45	-	32	4D 77	M	32	6D 109	m
	0E 14	SO ^N	34	2E 46	.	31	4E 78	N	31	6E 110	n
	0F 15	SI ^O	08	2F 47	/	18	4F 79	O	18	6F 111	o
	10 16	DLE ^P	0B	30 48	0	19	50 80	P	19	70 112	p
	11 17	DC1 ^Q	02	31 49	1	10	51 81	Q	10	71 113	q
	12 18	DC2 ^R	03	32 50	2	13	52 82	R	13	72 114	r
	13 19	DC3 ^S	04	33 51	3	1F	53 83	S	1F	73 115	s
	14 20	DC4 ^T	05	34 52	4	14	54 84	T	14	74 116	t
	15 21	NAK ^U	06	35 53	5	16	55 85	U	16	75 117	u
	16 22	SYN ^V	07	36 54	6	2F	56 86	V	2F	76 118	v
	17 23	ETB ^W	08	37 55	7	11	57 87	W	11	77 119	w
	18 24	CAN ^X	09	38 56	8	2D	58 88	X	2D	78 120	x
	19 25	EM ^Y	0A	39 57	9	2C	59 89	Y	2C	79 121	y
	1A 26	SUB ^Z	34	3A 58	:	15	5A 90	Z	15	7A 122	z
01	1B 27	Esc ^[	33	3B 59	;		5B 91	[		7B 123	{
	1C 28	FS ^\	2B	3C 60	<		5C 92	\		7C 124	
	1D 29	GS ^]	0B	3D 61	=		5D 93	]		7D 125	}
	1E 30	RS ^^	2B	3E 62	>	29	5E 94	^		7E 126	~
	1F 31	US ^_	0C	3F 63	?	35	5F 95	_	53	7F 127	DEL

Challenge 22

Überlege zuerst mit Hilfe der Tabelle und **überprüfe danach** folgenden Aussagen mit **Vergleichs-Operatoren**:

1. "Test" == "test"
2. "und" > "&"
3. "-2" < "2"
4. "(Text)" > "{Text}"

Hier kommt deine Lösung hin.

Lösung

```
print("Test"=="test")
print("und"> "&")
print("-2"<"2")
print("(Text)">"{Text}")
```

False
True
True
False

Playground

```
print(7 == 8) # False
print(7 != 8) # True
print(1 < 4 < 7) # True
```

Mehrere Variablen gleichzeitig vergleichen

```
a, b, c = 1, 2, 3
print((a, b, c) == (1, 2, 3)) # True
print((a, b, c) == (2, 1, 3)) # False
```

String-Vergleich

```
print('a' < 'b') # True
print('a' < 'A') # False
print('A' < 'a') # True
print('Hello' > 'Hallo') # True
```

```
print(7 < 9 > 4) # True
print(9 > 4) # True
```

```
print(7 < 9 > 11) # False
print(7 < 9) # False
```

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel F: Logische Operatoren

Logische Operatoren

Logische Operatoren verknüpfen Elemente bei Vergleichen und geben immer einen **Wahrheitswert** zurück.

Sie werden **als Worte** ausgeschrieben.

Operator	Beschreibung	Bedingung
<i>and</i>	was links steht UND was rechts steht	BEIDE müssen stimmen
<i>or</i>	was links steht ODER was rechts steht	EINER der beiden MUSS stimmen
<i>not</i>	NICHT das, was folgt	der Wahrheitswert der folgenden Aussage wird umgekehrt
<i>in</i>	was links steht ist IN dem, was rechts steht	die Objekte müssen iterable sein (Strings, Listen, ...)
<i>is</i>	IST genau das gleiche Objekt im Speicher	die zugewiesene ID des Objekt muss exakt übereinstimmen

```
print(1<2 and 2<3)
print(1<0 or 2>1)
print(not True)
print(not 1<0 or 2>1)
print(not( 1<0 or 2>1))
print(not 1<2 and 2<3)
```

```
True
True
False
True
False
False
```

```
print("e" in "hello")
print ("e" in "hallo")
print("e" in "hello" and "hallo") #Achtung!!!
print(1==1 and 2)
print(1==3 and 2) #False
```

```
True
False
hallo
2
False
```

```
text1="text"
text2="text"

# ist text1 das gleiche Objekt wie text2
print(text1 is text2)

print(id(text1))
print(id(text2))

True
140714429928672
140714429928672
```

Challenge 23

Notiere dir jeweils den **erwarteten Output** des folgenden Codes und **überprüfe** diesen dann.

```
one = False
b = True
hello = not(one)
test = not(b)

print(not one and test) # 1
print(hello and test or one and b) # 2
print(hello is one) # 3
print(test is b or hello and b) # 3
print(not b is hello or one) # 5
print(hello or test or b) # 6
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel G: Bedingungen (if, else, elif)

if

Mit **if** kannst du **prüfen**, ob eine bestimmte **Bedingung wahr oder falsch** ist.

Es muss **immer** eine **Operation** angegeben werden, die sich auf **wahr oder falsch** beläuft.

Wenn die Bedingung erfüllt, also wahr, ist, wird der eingeschobene Code darunter ausgeführt.

```
num = 10
if num == 10:
    print("ich bin wahr")
```

ich bin wahr

```
clock = 9
```

```
if clock >= 10:
    print("PAUSE!")
else:
    print("Ich will pause!")
```

Ich will pause!

else

else wird **dann** ausgeführt, **wenn** ein zuvorstehendes **if nicht** zutrifft.

Es bezieht sich **immer** auf das **if vor ihm** und ist **optional**.

Es benötigt **keine Bedingung**, da es immer als Alternative ausgeführt wird.

Nur das if oder das else wird ausgeführt, nie beide.

```
num ="10"
if num ==10:
    print("Ich bin 10")
else:
    print("Ich bin was anderes")
```

Ich bin was anderes

elif

Mit **elif** kannst du **verschiedene Bedingungen nacheinander prüfen**.

Sobald **eine Bedingung wahr** ist und ihr Code ausgeführt wird, werden die **restlichen elif Bedingungen übersprungen**.

```
num =12
if num ==10:
    print("Ich bin 10")
```

```
elif num==11:
    print("Ich bin 11")
elif num>10:
    print("Ich bin grösser als 10")
else:
    print("Ich bin was anderes")
```

Ich bin grösser als 10

```
num =10
if num <=5:
    print("Zahl ist höchstens 5")
elif num <=100:
    print("Zahl ist höchstens 100")
elif num <=10:
    print("Ich werde nie ausgeführt")
```

Zahl ist höchstens 100

Challenge 24

Schreibe ein Programm mit Bedingungen, was bei der **Eingabe des Wetters** vorschlägt, **welche Kleidung** man mitnehmen sollte.

Wenn es regnet, sollte man eine Regenjacke mitnehmen.

Wenn es schneit, sollte man Stiefel mitnehmen.

Wenn es sonnig ist, sollte man leichte Kleidung einpacken.

Sonst ist es egal, was man einpackt.

```
weather = input("Gib das Wetter an: Regen, Schnee, Sonne")
if weather.lower()=="regen":
    print("Nimm eine Regenjacke mit!")
elif weather.lower()=="schnee":
    print("Nimm Stiefel mit!")
elif weather.lower()=="sonne":
    print("Pack leichte Kleidung ein!")
else:
    print("Es ist egal was du einpackst, aber denk an deine Pausen!")
```

Nimm eine Regenjacke mit!

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel H: While Loops

While Loop

Mit **while** können wir etwas **so lange ausführen**, wie die **Bedingung dahinter wahr** ist. Es **funktioniert wie ein if**, nur dass es so lange **wiederholt** wird, **bis die Bedingung nicht mehr wahr** ist.

ACHTUNG: Endlosschleifen können entstehen!

```
counter=1
while counter<=10:
    print(counter)
    counter+=1 #counter=counter+1
print("Ende")
```

```
1
2
3
4
5
6
7
8
9
10
Ende
```

```
counter=int(input("Gib eine Zahl ein"))
```

```
while counter >0:
    print("#"*counter)
    counter -=1
```

```
#####
####
###
##
#
```

Challenge 25

Finde mit Hilfe von **while** eine **Lösung** für die **folgenden Probleme**:

1. Finde heraus, wie viele Ziffern eine eingegebene Zahl hat.
2. Drehe die eingegebene Zahl um (letzte Ziffer als erste, etc.).

```

zahl=int(input("Geben sie eine natürliche Zahl ein"))
temp=zahl
ziffern=0
while temp>0:
    temp //=10
    ziffern +=1
print(f"Die Zahl {zahl} hat {ziffern} Ziffern")

```

Die Zahl 456 hat 3 Ziffern

Deine Lösung für Aufgabe 1.

```

import math
zahl=int(input("Geben sie eine natürliche Zahl ein"))
ziffern=1+int(math.log(zahl)/math.log(10))
print(ziffern)

```

4

Deine Lösung für Aufgabe 2.

```

zahl=int(input("Geben sie eine natürliche Zahl ein"))
temp=zahl
ziffern=0
while temp>0:
    temp //=10
    ziffern +=1
print(f"Die Zahl {zahl} hat {ziffern} Ziffern")

```

```

temp2=zahl
rewind=0
for i in range(ziffern):
    rewind += temp2//(10**(ziffern-i-1))*10**i
    temp2 -=temp2//(10**(ziffern-i-1))*10**(ziffern-i-1)
    print(rewind)
print("Rückwärts:",rewind)

```

'''

```

temp2=zahl
rewind=0
for i in range(ziffern):
    ziffer = temp2 // (10 ** (ziffern - i - 1))
    rewind = rewind * 10 + ziffer
    temp2 = temp2 % (10 ** (ziffern - i - 1))

```

```

temp2=zahl
rewind=0
for i in range(ziffern):
    ziffer = temp2 // (10 ** (ziffern - i - 1))
    rewind = rewind * 10 + ziffer
    temp2 = temp2 % (10 ** (ziffern - i - 1))

```

```

print(f"Die Zahl {zahl} rückwärts ist: {rewind}")
'''
temp2=zahl
rewind=0
while temp2!=0:
    digit = temp2 % 10
    temp2 //= 10
    rewind = rewind*10 + digit
print(f"Die Zahl {zahl} rückwärts ist: {rewind}")

```

```

Die Zahl 123456789 hat 9 Ziffern
1
21
321
4321
54321
654321
7654321
87654321
987654321
Rückwärts: 987654321
Die Zahl 123456789 rückwärts ist: 987654321

```

Lösung

While True, Break & Continue

Mit **while True** können wir **absichtlich** eine **Endlosschleife** erstellen.

Diese wird so lange **wiederholt**, **bis** wir in der Schleife eine **explizite Ausstiegsbedingung** erreichen: | Code | Beschreibung | | ---- | ----- | | **break** | **beendet** die **Schleife** und führt mit dem folgenden Code fort. | | **continue** | **beendet** den **aktuellen Durchlauf** der Schleife und **beginnt** einen **neuen**. |

Tipp: Man kann beide **auch** in **for-Loops** verwenden.

```

while True:
    print("#")
    break

my_string=''
while True:
    if my_string=="###":
        my_string += '*'
        continue
    if my_string == '###*':
        break

```

```
my_string += '#'
print(my_string)
print(my_string)
```

```
#
##
###
###*
```

Else nach Loops

Mit *else* können wir Code **nach dem Verlassen** der Schleife ausführen.
Wenn die **Schleife mit *break*** verlassen wird, wird auch der *else* **Block nicht ausgeführt**.

Tip: Man kann dies auch in *for-Loops* verwenden.

```
num = 10

while num >= 0:
    print(num)
    num -= 1
else:
    print("Countdown fertig!")
```

```
10
9
8
7
6
5
4
3
2
1
0
Countdown fertig!
```

Challenge 26

Passe die gegebene Loop **an**, so dass sie den **folgenden Anforderungen** entspricht:

1. Sobald die Schleife erfolgreich beendet ist, soll eine entsprechende Nachricht ausgegeben werden.
2. Wenn wir eine durch 10 teilbare Zahl erreichen, beenden wir die Schleife.

```
num = int(input("Gib eine Zahl ein:\n"))

while num > 0:
    if num % 2 == 0:
        print(f"{num} ist gerade.")
```

```
    num -= 3
else:
    print(f"{num} ist ungerade.")
    num -= 5
```

25 ist ungerade.
20 ist gerade.
17 ist ungerade.
12 ist gerade.
9 ist ungerade.
4 ist gerade.
1 ist ungerade.

Lösung

```
num = int(input("Gib eine Zahl ein:\n"))

while num > 0:
    if num % 2 == 0:
        print(f"{num} ist gerade.")
        num -= 3
    else:
        print(f"{num} ist ungerade.")
        num -= 5
    if num % 10 == 0 and num != 0:
        print(f"Zahl {num} durch 10 teilbar!")
        break
else:
    print("Schleife erfolgreich normal beendet!")
```

8 ist gerade.
5 ist ungerade.
Schleife erfolgreich normal beendet!

Playground

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel I: For Loops

For Loop

Mit *for* können wir Elemente eines Objekts durchlaufen.

Die Schleife **wiederholt** sich so lange, **bis** das **Ende des Objekts** erreicht ist.
Dabei ist in **jedem Durchlauf** der **aktuelle Index** des Objekts unter der **angegebenen Variable** gespeichert.

Der Name der **Variable** ist **frei wählbar**, aber Konvention ist *i* für **Index**.

```
my_string="I'm a string!"
for i in my_string:
    print(i)
```

I
,
m

a

s
t
r
i
n
g
!

```
my_string="I'm a string!"
```

```
for i in my_string:
    print(i)
```

```
my_list=[1,2,3,4,5]
```

```
for number in my_list:
    print(number)
```

```
for _ in my_list:    # kein zugriff auf die variable es wird nur gezählt
    print('Hi')
```

I
,
m

a

s
t
r

```
i
n
g
!
1
2
3
4
5
Hi
Hi
Hi
Hi
Hi
```

```
for number in range(10):
    print(number)
```

```
for number in range(1,10,3):
    print(number)
```

```
0
1
2
3
4
5
6
7
8
9
1
4
7
```

Challenge 27

Nutze eine **for-Loop** um die **Größe** des vorhandenen **Rechtecks** zu **verdoppeln**.

Tipp: Variablen können hier helfen.

```
#####
#       #
#       #
#####
```

Lösung

```
for i in range(18):
    print("#" ,sep=' ',end='')
print()
for j in range(6):
```

```

    print('#'+ ' '*16+'#',sep='')
for i in range(18):
    print("#",sep='',end='')

```

```

#####
#           #
#           #
#           #
#           #
#           #
#           #
#####

```

```

groesse = int(input("Schreib die gröÙe von rechteck in Zahl von #"))
print("Dein Rechteck ist so:")
for number in range(groesse):
    print("#", end="")
print()
for number in range(2):
    print("#" + " " * (groesse-2) + "#")

for number in range(groesse):
    print("#", end="")

```

```

print("Jetzt verdopple ich ihn dir:")
groesse = 2*groesse
for number in range(groesse):
    print("#", end="")
print()
for number in range(2):
    print("#" + " " * (groesse-2) + "#")

for number in range(groesse):
    print("#", end="")

```

```

for i in range(5):
    if i%3==2:
        continue
    if i%2==0:
        print("#####*2)
    else:
        print("#"+ ' '*2+"#")

```

```

#####
#           #
#           #
#####

```

```

# Hier hast du Platz, dich auszutoben
my_list = [[1, 2, 3], [1, 2, 3]]

```

```

for outer in range(len(my_list)):

```

```
for inner in range(len(my_list[0])):
    print(my_list[outer][inner])
```

Playground

```
nums = [1, 2, 3]
vals = nums
del vals[1:2]

print(id(nums))
print(nums)
print(id(vals))
print(vals)

my_string = list("Hello")
another_string = my_string
my_string = list("Bye")

print(id(my_string))
print(my_string)
print(id(another_string))
print(another_string)
```

Hier hast du Platz, dich auszutoben

Welt 2: Wald

Kapitel J: Randomisation

Random Modul

Das **random Modul** ist **in-built**, muss also **nicht zusätzlich installiert** werden.
Es ist aber **nicht automatisch aktiv**, muss also bei Bedarf **importiert** werden.

Methode	Beschreibung
<i>randrange(X, Y)</i>	gibt einen ganzen Wert zwischen <i>X</i> und <i>Y</i> zurück ohne Ende.
<i>randint(X, Y)</i>	gibt einen ganzen Wert zwischen <i>X</i> und <i>Y</i> zurück mit Ende.
<i>random()</i>	gibt einen Wert zwischen 0 und 1 zurück.(1 exklusive)
<i>uniform(X, Y)</i>	gibt einen Dezimal-Wert zwischen <i>X</i> und <i>Y</i> zurück.
<i>choice(X, Y, Z, ...)</i>	wählt einen Wert aus den übergebenen Werten aus.

Methode	Beschreibung
<i>shuffle(X)</i>	bringt die übergebenen Werte in eine zufällige Reihenfolge, wobei <i>X</i> ein iterable Objekt sein muss (z.B. Liste).

```
import random

print(random.randrange(1,10))  # 10 exklusiv
print(random.randint(1,10))    # 10 inklusive
print(random.random())
print(random.uniform(1,6))
print(random.choice(['Hallo', 'Bye', 'Hi', 'Ciao']))
print(random.choice('ABCDEFG'))
my_list=['Hallo', 'Bye', 'Hi', 'Ciao']
print(random.shuffle(my_list))
print(my_list)
```

3
10
0.1832311634280741
3.9167638359998413
Bye
g
None
['Hallo', 'Hi', 'Bye', 'Ciao']

Challenge 28

Für einen kleinen Shop wollen wir Rabatte einführen.

Hierzu geht ihr wie folgt vor:

1. Es wird entschieden, ob überhaupt ein Rabatt gegeben wird (50% Chance).
2. Wenn es einen Rabatt gibt, soll dieser zufällig zwischen 10% und 40% liegen.
3. Der reduzierte Preis des Produkts wird dann mit dem entsprechenden Rabatt ausgegeben.
4. Wenn kein Rabatt gegeben wird, wird auch dies mit dem normalen Preis ausgegeben.
5. Optional: Füge Preise für mehrere Produkte ein, wovon immer eins zufällig ausgewählt wird.

```
import random
```

```
product_cost = 10.0
```

Lösung

Seeds & "echter" Zufall

Es gibt **keinen echten Zufall** in Python (und bei Computern generell).

Jeder Zufall beruht auf einem **Seed**.

Wenn **dieser gleich** ist, werden **immer die exakt gleichen "zufälligen" Ergebnisse** erzielt.

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Hoehlen

Kapitel A: Module Importieren

Python Module

Python kommt mit **vielen Modulen built-in**.

Diese sind **Kernelemente von Python**, aber **zur Optimierung ausgelagert** und werden nur **bei Bedarf** mit **import** geladen.

Eine **vollständige Liste** aller integrierten Module und deren **Dokus** findest du **hier**:

<https://docs.python.org/3/py-modindex.html>

PyPi, pip & externe Module

Neben den Modulen, die immer mitgeliefert werden, gibt es für Python **tausende von der Community erstellte Module**.

Es gibt für (nahezu) **jeden Verwendungszweck** von Python **ein Modul**, was einem Arbeit abnimmt, neues ermöglicht oder Workflows anpasst.

ABER: zum **Lernen** ist das **Verlassen auf Module**, die man selber nicht versteht, **eher hinderlich**.

Code Minecraft With Python in 57 Seconds:

<https://www.youtube.com/watch?v=tONNC-bWSFM>

PyPi (Python Package Index):

<https://pypi.org/>

Packages mit pip einfach installieren:

<https://packaging.python.org/en/latest/tutorials/installing-packages/#use-pip-for-installing>

```
# Modul zum anfragen und verarbeiten von API-Requests
```

```
# pip install requests
```

```
# Modul zum erstellen von Plots zur Visualisierung von Daten
```

```
# pip install matplotlib
```

Welt 3: Hoehlen

Kapitel B: Math Modul

Math Basics

Das **Math Modul** hat viele verschiedene Methoden, die diverse **mathematische Operationen vereinfachen**.

Für uns ist erstmal nur **eine kleine Auswahl relevant**:

Methode	Beschreibung
<i>math.ceil(X)</i>	rundet X zur nächsten Ganzzahl auf .
<i>math.floor(X)</i>	rundet X zur nächsten Ganzzahl ab .
<i>math.pow(X, Y)</i>	exponentiert X mit Y .
<i>math.sqrt(X)</i>	zieht die Wurzel aus X .
<i>math.fmod(X, Y)</i>	gibt den Rest der Division von X durch Y zurück.
<i>math.factorial(X)</i>	bildet die Fakultät von X .
<i>math.gcd(X, Y, Z, ...)</i>	gibt den größten gemeinsamen Teiler von beliebig vielen Zahlen zurück.
<i>math.fsum(ITERABLE)</i>	gibt die Summe aller Elemente eines iterable Objekts zurück.
<i>math.prod(ITERABLE)</i>	gibt das Produkt aller Elemente eines iterable Objekts zurück.

Alle Methoden des **Math Moduls** findest du **hier**:

<https://docs.python.org/3/library/math.html>

Challenge 29

Nutze das **Math Modul**, um die **folgenden Rechnungen abzubilden**:

1. $1 * 2 * 3 * 4 * 5$

2. $100 / 10$

3. $2^{**}8$

4. $25 \% 8$

5. $1 + 3 + 8 + 11 + 28 + 13 + 4$

6. $5 * 24 * 9 * 11$

```
import math
```

```
# Hier kommt deine Lösung hin
```

Lösung

Playground

```
# Alle Methoden des Math Moduls findest du hier  
# https://docs.python.org/3/library/math.html
```

```
# Hier hast du Platz, dich auszutoben
```

Welt 3: Höhlen

Kapitel C: Binary & Bitwise Operatoren

Binary & Hex in Python

Wir können in Python auch mit **binären Zahlen** arbeiten.
Sie werden **im Speicher** jedoch **als Integers** hinterlegt und lassen sich daher auch **genau wie Integers behandeln**.

```
num = 0b0101 #binär  
#0b binär  
#0x Hex  
#0o Octal
```

```
print(num)  
print(type(num))
```

```
print(bin(num))  
print(hex(num))  
print(oct(num))
```

```
print(num+3)  
print(bin(num+3))
```

```
5  
<class 'int'>  
0b101  
0x5  
0o5  
8  
0b1000
```

Bitwise Operatoren

Es gibt bestimmte **Operatoren**, die mit **Variablen auf binärer Ebene** interagieren: |
Operator | **Bedeutung** | **Beschreibung** | | ----- | ----- | ----- | | **&** | **AND** | **setzt**
jeweils das Bit **auf 1**, **wenn beide** Bits 1 sind. | | **|** | **OR** | **setzt** jeweils das Bit **auf 1**, **wenn**
mindestens eines der Bits 1 ist. | | **^** | **XOR** | **setzt** jeweils das Bit **auf 1**, **wenn nur eines**
der Bits 1 ist. | | **~** | **NOT** | **kehrt alle Bits um**. | | **<< X** | **Left Shift** | **verschiebt** alle Bits
X Schritte nach links. Wie **multiplizieren** mit **2**X**. | | **>> X** | **(Signed) Right Shift** |
verschiebt alle Bits **X Schritte nach rechts**. Wie **Floor-Division** mit **2**X**. |

```
num1=0b0101  #5
num2=0b1101  #13

#output
print(num1,num2)
print(bin(num1),bin(num2))

# AND
print(num1&num2)
print(bin(num1&num2))

#OR
print(num1 | num2)
print(bin(num1 | num2))

# XOR
print(num1 ^ num2)
print(bin(num1 ^ num2))

# NOT
print(~num1 )
print(bin(~num1 ))

# Shifting
print(bin(num1))
print(num1 << 1)    # mal 2^1 (<<n 2^n)
print(bin(num1 << 1))

print(num1 >> 1)
print(bin(num1 >> 1)) # integer durch 2^1 (>>n //2^n)

5 13
0b101 0b1101
5
0b101
13
0b1101
8
0b1000
-6
-0b110
```

```
0b101
10
0b1010
2
0b10
```

Challenge 30

Führe die folgenden **binären Operationen** erst **händisch** durch und **implementiere** sie dann in Python.

Gib die **Ergebnisse** jeweils als **Dezimal-** und **Binärzahl** an.

1. 7 ODER 18
2. 10110110 UND 11
3. NUR 10100011 ODER 00110111

Hier kommt deine Lösung hin

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Höhlen

Kapitel D: String Methoden

String Methoden

Mit **Strings** können wir **durch Methoden** auch noch **spezifischer** interagieren. Diese **funktionieren nur**, wenn wir sie auf den **Datentyp Strings** anwenden.

Methode	Beschreibung
STRING.lower()	setzt alle Zeichen in lower case (Kleinbuchstaben).
STRING.upper()	setzt alle Zeichen in upper case (Großbuchstaben).
STRING.title()	setzt das erste Zeichen jedes Wortes in upper case , alles andere in lower case .
STRING.capitalize()	setzt das erste Zeichen des Strings in upper case , alles andere in lower case .
STRING.count(X)	gibt die Anzahl der Vorkommnisse von X zurück.
STRING.find(X)	gibt den Index des ersten Vorkommnisses von X zurück.

Methode	Beschreibung
<i>STRING.rfind(X)</i>	gibt den Index des ersten Vorkommnisses von X , von rechts angefangen, zurück .
<i>STRING.strip()</i>	entfernt alle Leerzeichen am Anfang und Ende .
<i>"SEP".join(STRINGS)</i>	fügt alle STRINGS (z.B. Liste mit Strings) mit SEP getrennt zusammen .

Hier findest du eine **Auflistung** aller **String Methoden**:

https://www.w3schools.com/python/python_ref_string.asp

Wir können **Strings** auch **auf** ihre **Inhalte überprüfen**, und uns entsprechend einen **Bool** **ausgeben** lassen.

Methode	Beschreibung
<i>STRING.islower()</i>	gibt True zurück, wenn alle Zeichen in lower case (Kleinbuchstaben) sind.
<i>STRING.isupper()</i>	gibt True zurück, wenn alle Zeichen in upper case (Großbuchstaben) sind.
<i>STRING.isdigit()</i>	gibt True zurück, wenn alle Zeichen in digits sind.
<i>STRING.isalpha()</i>	gibt True zurück, wenn alle Zeichen in im Alphabet sind.
<i>STRING.isalnum()</i>	gibt True zurück, wenn alle Zeichen in alphanumerisch sind.

Hier findest du eine **Auflistung** aller **String Methoden**:

https://www.w3schools.com/python/python_ref_string.asp

Challenge 31

Implementiere eine Lösung für folgende **Problemstellung**:

Für den Anmeldeprozess eines Event-Planers müssen Teilnehmende ihre Vor- und Nachnamen eingeben.

Es soll abgesichert werden, dass egal wie die Namen eingetippt werden, die Ausgabe immer Vor- und Nachnamen mit jeweils beginnendem Großbuchstaben ist. Schließlich soll eine gute lesbare Ausgabe mit dem korrekt formatieren Namen gemacht werden.

Extra Challenge: Es wird nicht seperat nach Vor- und Nachnamen gefragt, sondern in einer einzelnen Eingabe.

```
first_name = input("Gib deinen Vornamen ein:\n")
last_name = input("Gib deinen Nachnamen ein:\n")

name = input("Gib deinen Vor- & Nachnamen ein:\n")
```

Lösung

```
name = input("Gib deinen Vor- & Nachnamen ein:\n").split()
name = " ".join(name).lower().title()
```

```
print(f"Vielen Dank für deine Anmeldung, {name}!")
```

Vielen Dank für deine Anmeldung, Hans Peter!

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Höhlen

Kapitel E: Match Case

Match Case Basics

Match Cases funktionieren **ähnlich wie** *if* Bedingungen, aber bieten den **Vergleich** von **beliebig vielen Werten** innerhalb einer Bedingung.

Sie sind dann besonders **nützlich**, wenn man **mehrere Werte einer Variable erwartet** und bei **jedem Wert etwas anderes** passieren soll.

Ähnlich wie *else* kann auch hier mit *****case _* für alle anderen Fälle**** etwas getan werden.

Challenge 32

Benutze **Match Cases**, um für den **einggegebenen Wochentag** eine **entsprechende emotionale Nachricht** auszugeben.

Zum Beispiel:

Montag: "Oh no..."

Freitag: "Oh yay!"

Hier kommt deine Lösung hin

Lösung

Match Case Advanced

Match Cases können auch **mehrere Fälle verknüpfen**.

Hierzu verwenden wir **logische Operatoren**, welche im **match case** aber etwas anders **abgebildet** werden.

Challenge 33

Benutze **Match Cases**, um folgende **Anforderung abzubilden**:

Ein Unternehmen berechnet in seinem Shop beim Versand in bestimmte Länder verschiedene Versandkosten:

1. Innerhalb Deutschland: keine Versandkosten
 2. Frankreich, Niederlande, Schweiz, Österreich, Belgien, Polen: 2.00 Euro
 3. UK: 3.50 Eurp
 4. USA: 5.50 Euro
 4. Für alle anderen EU Länder: 3.00 Euro
 5. Für alle anderen nicht-EU Länder: 7.00 Euro
- Nutzende sollen ihr Land (auch EU) eingeben können.

Hier steht der nötige Basiscode für die Aufgabe

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Höhlen

Kapitel F: Tupel

Tupel erzeugen

Tupel funktionieren **fast genau wie Listen**, aber sie sind nach dem Erstellen **komplett unveränderbar**.

Tupel sind immutable (unveränderbar).

Tupel Klammern

```
my_tupel1 = ("rot", "blau", "grün")
print(my_tupel1)
print(type(my_tupel1))
```

#Tupel Konstruktor

```
my_tupel2=tuple(("lila", "orange", "schwarz"))  #achtung doppelklammern
print(type(my_tupel2))
print(my_tupel2)
```

#Tupel in Liste

```
my_list=list(my_tupel2)
```

```

print(my_list)
print(type(my_list))

#List in Tupel
my_list2=[1,2,3,4,5,6,7,8,9]
my_tupel3=tuple(my_list2)
print(my_tupel3)
print(type(my_tupel3))

# Ein Element
my_tuple4=("weiß") #hier String!
print(type(my_tuple4))
print(my_tuple4)

my_tuple5=("weiß",) # mit komma wirds zum tupel
print(type(my_tuple5))
print(my_tuple5)
print(len(my_tuple5))

('rot', 'blau', 'grün')
<class 'tuple'>
<class 'tuple'>
('lila', 'orange', 'schwarz')
['lila', 'orange', 'schwarz']
<class 'list'>
(1, 2, 3, 4, 5, 6, 7, 8, 9)
<class 'tuple'>
<class 'str'>
weiß
<class 'tuple'>
('weiß',)
1

my_tupel1 = ("rot","blau","grün")
print(my_tupel1)
print(type(my_tupel1))
# my_tuple[0]="gelb" geht nicht
my_tupel1 = ("gelb","blau","grün") #jetzt gehts
print(my_tupel1)

('rot', 'blau', 'grün')
<class 'tuple'>
('gelb', 'blau', 'grün')

```

Challenge 34

Erstelle eine Liste mit **Tupel**, wo jedes **Tupel** eine Zahl von 1 bis 5 mit ihrem Quadrat enthält.

Hier kommt deine Lösung hin

Lösung

```

my_list3=[1,2,3,4,5]
ziel=[]
for i in my_list3:
    ziel.append(tuple((i,i**2)))

print(ziel)

[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

my_list=[]
for i in range (1,6):
    my_list.append((i, i**2))
print(my_list)

[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

ziel = [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

my_list = list(ziel[1])
print(my_list[1])

4

ziel = [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

# Tupel aus Liste extrahieren
# Dann als eigene Liste speichern
my_list = list(ziel[1])
print(my_list[1])

# Tupel mit Index geht
print(ziel[1][1])

```

Tupel anwenden

Aktion	Beschreibung
Indexing	Wir können auf jeden Index von Tupeln zugreifen und sie entsprechend auch slicen .
Unpacking	Tupel können auch entpackt werden, wobei jedes Element einer Variable zugewiesen wird.
Enumerate	Mit <i>enumerate(TUPEL)</i> in einer for-Loop werden Tupel als neue Tupel durchnummeriert .
Joining	Auch wenn Tupel selbst unveränderbar sind, können wir mehere Tupel zusammen als ein neues speichern.

```

my_tupel1 = ("rot","blau","grün")
my_tupel2=tuple(("lila","orange","schwarz"))

# Index ansprechen
print(my_tupel1[1])

```

```

print(my_tupel2[1:3])
print(my_tupel1[::-1])

# Tupel entpacken
colour1, colour2, colour3 =my_tupel1    #2 werte würden nicht gehen da tupel
                                     3d
print(colour1,colour2,colour3)

# Tupel mit enumerate()
for element in enumerate(my_tupel1):
    print(element)

#Tupel verbinden mit +
my_tuple7=my_tupel1+my_tupel2
print(my_tuple7)

```

```

blau
('orange', 'schwarz')
('grün', 'blau', 'rot')
rot blau grün
(0, 'rot')
(1, 'blau')
(2, 'grün')
('rot', 'blau', 'grün', 'lila', 'orange', 'schwarz')

```

Challenge 35

Nutze **Tupel** und **Listen**, um eine **Lösung** für die folgende **Problemstellung** zu implementieren:

Bei einem Supermarkt werden geplante Lieferung über die Woche in der Software mit Listen und Tupeln abgebildet.

Zur besseren Übersicht soll die Software nun wie folgt erweitert werden:

1. Jeweils eine Liste pro Wochentag, welche Produkte in welchen Mengen an diesem Tag ankommen.
2. Eine einfache Abfrage, wie viele Kisten Produkte insgesamt an einem Tag eintreffen.

Extra Challenge: versuche die Anpassungen so zu verallgemeinern, dass sie auf verschiedene Listen mit dem gleichen Aufbau angewendet werden können.

```

supplier1 = [("Montag", "Käse", 34), ("Mittwoch", "Zwiebeln", 19),
             ("Mittwoch", "Zwiebeln", 19), ("Montag", "Brot", 42)]
supplier2 = [("Montag", "Fisch", 15), ("Dienstag", "Fleisch", 23)]

```

Lösung

```

supplier1 = [("Montag", "Käse", 34), ("Mittwoch", "Zwiebeln", 19),
             ("Mittwoch", "Zwiebeln", 19), ("Montag", "Brot", 42)]
supplier2 = [("Montag", "Fisch", 15), ("Dienstag", "Fleisch", 23)]

```

```

liste_mo=[]
liste_di=[]
liste_mi=[]
liste_do=[]
liste_fr=[]
liste_sa=[]
liste_so=["Spinnsch?"]
suppliers = supplier1 + supplier2

```

```

for i in range(len(suppliers)):
    if suppliers[i][0]=='Montag':
        liste_mo.append(suppliers[i])
    elif suppliers[i][0]=='Dienstag':
        liste_di.append(suppliers[i])
    elif suppliers[i][0]=='Mittwoch':
        liste_mi.append(suppliers[i])
    elif suppliers[i][0]=='Donnerstag':
        liste_do.append(suppliers[i])
    elif suppliers[i][0]=='Freitag':
        liste_fr.append(suppliers[i])
    elif suppliers[i][0]=='Samstag':
        liste_sa.append(suppliers[i])

```

```

print(liste_mo)
print(liste_di)
print(liste_mi)
print(liste_do)
print(liste_fr)
print(liste_sa)
print(liste_so)

```

```

oberliste=[liste_mo,liste_di,liste_mi,liste_do,liste_fr,liste_sa]
counter=1
for liste in oberliste:
    summe=0
    for j in range(len(liste)):
        summe+=liste[j][2]
    print(counter,":",summe)
    counter+=1

```

```

#summe_mo=0
#for i in range(len(liste_mo)):
#    summe_mo+=liste_mo[i][2]
#print("Montag:",summe_mo)

```

```
[('Montag', 'Käse', 34), ('Montag', 'Brot', 42), ('Montag', 'Fisch', 15)]
[('Dienstag', 'Fleisch', 23)]
[('Mittwoch', 'Zwiebeln', 19), ('Mittwoch', 'Zwiebeln', 19)]
[]
[]
[]
['Spinnsch?']
1 : 91
2 : 23
3 : 38
4 : 0
5 : 0
6 : 0
```

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Höhlen

Kapitel G: Dictionaries

Dictionaries erzeugen

Dictionaries funktionieren ähnlich wie **Listen**, aber sie **verknüpfen** immer einen **Schlüssel (Key)** mit einem **Wert (Value)**.

Keys sind (fast) immer **Strings**, die den **Wert beschreiben**: z.B. könnte der Wert 27 durch den Key *"Alter"* beschrieben werden.

Ints, Floats & Tupel sind als **Keys** auch **möglich**.

Values können **jeglichen Datentyp** haben, auch Listen, oder sogar Dictionaries (nested dicts).

Es gibt verschiedene Möglichkeiten, **Dictionaries** zu erzeugen.

Dictionaries sind mutable (veränderbar).

key : value (Pair)

#Dictionary Klammern {}

```
my_dict1={"Name": "Sarah", "Alter": 34, "Beruf": "Köchin"}
print(my_dict1)
print(type(my_dict1))
```

```

# Staggered
my_dict2 = {
    "Name": "Nico",
    "Alter": 51,
    "Beruf": "Pilot"
}
print(my_dict2)
print(my_dict2["Name"])

# Konstruktor dict()
my_dict3=dict(name="Egon",alter=39,beruf="Lehrer")
print(my_dict3)

```

```

{'Name': 'Sarah', 'Alter': 34, 'Beruf': 'Köchin'}
<class 'dict'>
{'Name': 'Nico', 'Alter': 51, 'Beruf': 'Pilot'}
Nico
{'name': 'Egon', 'alter': 39, 'beruf': 'Lehrer'}

questions = [
    {
        "type": "multiple-choice",
        "question": "Was ist der Datentyp von `type(42)` in Python?",
        "options": [
            "int",
            "float",
            "type",
            "class"
        ],
        "correct": [2],
        "explanation": "a) Falsch. Das ist der Wert selbst.\nb) Falsch.\nc) Richtig. `type()` gibt immer ein type-Objekt zurück (die Klasse des Objekts).\nd) Falsch. 'class' ist ein Keyword, aber der Typ ist <class 'type'>.",
        "penalty": False
    },
    {
        "type": "multiple-choice",
        "question": "Welche Anweisung erzeugt eine leere Liste?",
        "options": [
            "list()",
            "{}",
            "set()",
            "tuple()"
        ],
        "correct": [0],
    }
]

```

```

        "explanation": "a) Richtig. `list()` oder `[ ]` erzeugen eine leere  

        Liste.\nb) Falsch. Das ist ein leeres Dictionary.\nc) Falsch.  

        Leeres Set.\nd) Falsch. Leeres Tuple.",
        "penalty": False
    },
    {
        "type": "multiple-choice",
        "question": "Was gibt `bool([ ])` zurück?",
        "options": [
            "True",
            "False",
            "None",
            "TypeError"
        ],
        "correct": [1],
        "explanation": "a) Falsch.\nb) Richtig. Leere Sequenzen (List, Tuple,  

        String, etc.) sind falsy in Python.\nc) Falsch.\nd) Falsch.",
        "penalty": False
    }
]

```

```

attribute=["Name","Alter","Beruf","Ort"]
values1=["Carl",29,"Musiker"]
values2=["Johan",22,"Schreiner","Freiburg","Hackbratenbrunnen"]

```

```

my_dict7=dict(zip(attribute,values1))
print(my_dict7)

```

```

my_dict8=dict(zip(attribute,values2))
print(my_dict8)

```

```

{'Name': 'Carl', 'Alter': 29, 'Beruf': 'Musiker'}
{'Name': 'Johan', 'Alter': 22, 'Beruf': 'Schreiner', 'Ort': 'Freiburg'}

```

Challenge 36

Nutze alle drei Möglichkeiten, um jeweils ein Dictionary mit den folgenden Daten und passenden Keys zu erstellen:

1. Paper, 325, International Paper, 1350.75
2. Pencil, 55, Faber Castell, 321.62
3. Printer Ink, 30, Epson, 287.36

Hier kommt deine Lösung hin

Lösung

```

my_dict4={"Artikel":"Paper","Menge": 325,"Hersteller":"International  

        Paper","Preis": 1350.75}
my_dict5=dict(artikel="Pencil",menge=55,hersteller="Faber  

        Castell",preis=321.62)
#my_dict6=dict(artikel="Printer",menge=30,hersteller="Epson",preis=287.36)

```

```

attribute=["Artikel","Menge","Hersteller","Preis"]
values=["Printer",30,"Epson",287.36]
my_dict6=dict(zip(attribute,values))
print(my_dict4)
print(my_dict5)
print(my_dict6)

{'Artikel': 'Paper', 'Menge': 325, 'Hersteller': 'International Paper',
'Preis': 1350.75}
{'artikel': 'Pencil', 'menge': 55, 'hersteller': 'Faber Castell', 'preis':
321.62}
{'Artikel': 'Printer', 'Menge': 30, 'Hersteller': 'Epson', 'Preis': 287.36}

```

Dictionaries manipulieren

Wir können **Dictionaries** auch **manipulieren**:

Code	Beschreibung
<i>len(DICT)</i>	gibt die Länge des Dictionary's aus.
<i>DICT.get(KEY)</i>	gibt das Value des Key's zurück.
<i>KEY in DICT</i>	gibt True zurück, wenn der Key vorhanden ist.
<i>DICT.update(DICT)</i>	fügt die Inhalte eines weiteren Dictionary's meinem vorhandenen hinzu .
<i>del DICT[KEY]</i>	löscht Key und Value .
<i>DICT.pop(KEY)</i>	löscht Key und Value und gibt das Value zurück .
<i>DICT.clear()</i>	löscht den Inhalt des Dictionary's, aber behalt es als Objekt bei.

```

my_dict1={"Name": "Sarah","Alter": 34, "Beruf": "Köchin"}
print(my_dict1)

```

```

# anzehal der key value-pairs
print(len(my_dict1))

```

```

#get()
print(my_dict1.get("Name"))

```

```

print(my_dict1.items())

```

```

#in geht nach key's
print("Alter" in my_dict1)

```

```

#values:
print(my_dict1.values())

```

```

#update(), hängt hinten an
my_dict1.update({"Straße": "Hackbrunnenstraße"})
print(my_dict1)

```

```

# del Löschen eines paares!
del my_dict1["Beruf"]
print(my_dict1)

# pop()
print(my_dict1.pop("Alter")) #bei der Liste wäre es der index und nicht der
                             key
print(my_dict1)

#clear()
#entfernt alles
my_dict1.clear()
print(my_dict1)

{'Name': 'Sarah', 'Alter': 34, 'Beruf': 'Köchin'}
3
Sarah
dict_items([('Name', 'Sarah'), ('Alter', 34), ('Beruf', 'Köchin')])
True
dict_values(['Sarah', 34, 'Köchin'])
{'Name': 'Sarah', 'Alter': 34, 'Beruf': 'Köchin', 'Straße':
'Hackbrunnenstraße'}
{'Name': 'Sarah', 'Alter': 34, 'Straße': 'Hackbrunnenstraße'}
34
{'Name': 'Sarah', 'Straße': 'Hackbrunnenstraße'}
{}
{}

```

Challenge 37

Manipuliere das gegebene **Dictionary** um die **folgenden Outputs** zu bekommen:

1. True
2. 3
3. 5
4. Peter
5. False Yes
6. {}

```

my_dict = {"Key1": False, "Key2": "Yes", "Key3": 1.5, "Key4": [1, 2, 3],
           "Key5": {"Name": "Peter"}}

```

Lösung

```

my_dict = {"Key1": False, "Key2": "Yes", "Key3": 1.5, "Key4": [1, 2, 3],
           "Key5": {"Name": "Peter"}}
print(not my_dict.get("Key1"))
print(my_dict.get("Key4")[2])
print(my_dict.get("Key4")[2]+my_dict.get("Key4")[1])
print(my_dict.get("Key5")["Name"])
print(my_dict.get("Key1"),my_dict.get("Key2"))

```

```
my_dict.clear()
print(my_dict)
```

```
True
3
5
Peter
False Yes
{}
```

Playground

Hier hast du Platz, dich auszutoben

Welt 3: Höhlen

Kapitel H: Sets

Sets erzeugen

Sets funktionieren **ähnlich wie Listen**, aber sind **unsortiert**, **nicht-indexed** und erlauben **keine Dopplungen**.

Im Gegensatz zu Listen können wir **Set-Items** also **nicht durch Indexe** ansprechen. Die **Reihenfolge** ist **arbiträr**, folgt also nicht derer, die wir bei der Erstellung angeben.

Es gibt **verschiedene Möglichkeiten**, **Sets** zu erzeugen.

Sets sind mutable (veränderbar).

Challenge 38

Implementiere eine Lösung für folgende Problemstellung, indem du **ein Set** mit **allen vorhandenen Allergien** erstellst:

Für eine Firmenfeier soll abgefragt werden, gegen was Teilnehmende allergisch sind.

Hierzu sollen alle relevanten Allergien festgehalten werden, damit das Buffet korrekt zubereitet und gelabelt werden kann.

Die Anzahl, wie oft Teilnehmende an einer bestimmten Allergie leiden, ist hierfür irrelevant - nur, ob sie überhaupt vorkommt.

```
participant1 = ["Laktose", "Weizen"]
participant2 = ["Ei"]
participant3 = ["Erdnuss", "Soja", "Weizen"]
```

```
participant4 = ["Schalentiere", "Ei"]
participant5 = ["Erdnuss", "Soja", "Laktose", "Schalentiere"]
```

Lösung

Sets manipulieren

Wir können **Sets** auch **manipulieren**:

Code	Beschreibung
<code>len(SET)</code>	gibt die Länge des Set's aus.
<code>VALUE in SET</code>	gibt True zurück, wenn <code>VALUE</code> vorhanden ist.
<code>SET.add(VALUE)</code>	fügt <code>VALUE</code> dem Set hinzu .
<code>SET.remove(VALUE)</code>	entfernt <code>VALUE</code> aus dem Set.
<code>SET.discard(VALUE)</code>	entfernt <code>VALUE</code> aus dem Set.
<code>SET.pop()</code>	entfernt ein zufälliges Element aus dem Set.
<code>SET.clear()</code>	entfernt alle Elemente aus dem Set.

Challenge 39

Übernehme deinen Code aus der vorherigen Challenge.

Aufgrund verspäteter Anmeldung sind noch zwei weitere Teilnehmende mit Allergien hinzugekommen.
Deren Allergien müssen auch berücksichtigt werden.
Zur Vorbereitung der Buffet-Anrichtung muss ebenfalls die gesamte Menge der verschiedenen Allergien bekannt sein.

```
participant6 = ["Schalentiere", "Weizen", "Sesam"]
participant7 = ["Laktose", "Fisch"]
```

Lösung

Sets Joinen

Sets eignen sich besonders gut zum **Abgleich** und **manipulieren** von **Schnittmengen**:

ACHTUNG: die **Alternative**, meist Kurzschreibweisen, funktionieren **nur**, wenn **alle** verwendeten **Datentypen Sets** sind!

Methode	Alternative	Beschreibung
<code>NEWSET = SET1.union(SET2, SET3, ...)</code>	<code>SET1 SET2</code>	fügt alle angegeben Sets zusammen und gibt ein neues Set zurück.

Methode	Alternative	Beschreibung
<i>SET1.update(SET2)</i>	KEINE	fügt <i>SET1</i> alle Elemente von <i>SET2</i> hinzu.
<i>NEWSET = SET1.intersection(SET2)</i>	<i>SET1 & SET2</i>	gibt ein neues Set mit allen Elementen zurück, die sowohl in <i>SET1</i> als auch <i>SET2</i> sind.
<i>SET1.intersection_update(SET2)</i>	KEINE	entfernt alle Elemente aus <i>SET1</i> die nicht auch in <i>SET2</i> sind.
<i>NEWSET = SET1.difference(SET2)</i>	<i>SET1 - SET2</i>	gibt ein neues Set mit allen Elementen zurück, die nur in <i>SET1</i> aber nicht <i>SET2</i> sind.
<i>SET1.difference_update(SET2)</i>	KEINE	entfernt alle Elemente aus <i>SET1</i> die auch in <i>SET2</i> sind.
<i>NEWSET = SET1.symmetric_difference(SET2)</i>	<i>SET1 ^ SET2</i>	gibt ein neues Set mit allen Elementen zurück, die entweder nur in <i>SET1</i> oder nur in <i>SET2</i> sind.
<i>SET1.symmetric_difference_update(SET2)</i>	KEINE	entfernt alle Elemente aus <i>SET1</i> die auch in <i>SET2</i> sind und fügt hinzu, welche nur in <i>SET2</i> sind.

Challenge 40

Übernehme deinen Code aus der vorherigen Challenge.

Bei der großen Jahresfeier kommen viele verschiedene Abteilungen der Firma zusammen.

Auch hier wird ein Buffet geplant, wo Allergien bekannt sein müssen. Erstelle ein Set, was die Allergien deiner, als auch aller anderen Abteilungen zusammenführt.

Da die Firma gerne Daten verarbeitet und nicht allzu viel Wert auf Datenschutz legt, soll nun zusätzlich eine Liste mit Allergien erstellt werden, die in allen Abteilungen vorkommen und eine mit Allergien, die jeweils nur in einer Abteilung vorkommen.

```
allergies_it = allergies
allergies_hr = {"Weizen", "Ei", "Soja", "Laktose"}
allergies_marketing = {"Fisch", "Laktose", "Weizen", "Sesam", "Soja"}
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 4: Lichtung

Kapitel A: Funktionen

Funktionen Basics

Funktionen sind das **Herzstück** vieler Programme.

Sie erlauben uns das **modulieren**, bzw. **auslagern** von Operationen, so dass diese **nur aufgerufen** werden, **wenn** sie direkt **benötigt** werden.

Außerdem können wir so den **Code wiederverwenden** wann immer er benötigt wird.

Return

Mit **return geben** wir eine **Variable** aus einer Funktion **zurück**.

Diese kann dann **im ganzen Programm weiterverwendet** werden.

Sobald ein **return** eintrifft wird die **Funktion beendet**.

Challenge 41

Nutze **Funktionen** um eine **Lösung** für folgende **Problemstellung** zu **implementieren**:

Eine Liste mit einer beliebigen Anzahl von Zahlen wird übergeben.

Daraus soll die Summe aller Zahlen in der Liste gebildet und ausgegeben werden.

Teste deine Funktion mindestens mit diesen Listen

```
list1 = [1, 8, 2, 5, 11]
```

```
list2 = [-5, 25, 11, -9]
```

```
list3 = [112.72, 52.42, 251.256, 191.918]
```

Lösung

```
def bestimme_summe(list):  
    summe=0  
    for i in range(len(list)):  
        summe += list[i]  
    return summe
```

```
list1 = [1, 8, 2, 5, 11]
```

```
list2 = [-5, 25, 11, -9]
```

```
list3 = [112.72, 52.42, 251.256, 191.918]
```

```
print(bestimme_summe(list1))
print(bestimme_summe(list2))
print(bestimme_summe(list3))
```

27

22

608.314

Scope & Global

Alles, was **in einer Funktion** deklariert wird, ist auch **nur innerhalb der Funktion gültig**.

Wir können **Variablen** mit *global* deklarieren, um sie **auch außerhalb der Funktion** zu verwenden.

Challenge 42

Nutze **Funktionen** um eine **Lösung** für folgende Problemstellung zu **implementieren**. **Probiere** eine Lösung **mit und** eine **ohne *global*** aus.

Dein kleines Unternehmen braucht einen simplen Mehrwertsteuerrechner, der einen Preis annimmt, und ob das Produkt Grundbedarf ist.

Beim Grundbedarf wird die ermäßigte Mehrwertsteuer (7%) aufgeschlagen, andernfalls die normale (19%).

Die Ausgabe soll den entsprechenden Preis mit Mehrwertsteuer und den Mehrwertsteuersatz ausgeben.

Hier kommt deine Lösung hin

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 4: Lichtung

Kapitel B: Rekursion

Rekursive Funktionen

Rekursion bedeutet, eine Funktion **innerhalb von sich selbst** erneut aufzurufen.

Rekursionen haben immer einen **Base Case** und einen **Recursive Case**:

- **Base Case:** verhindert Endlosschleifen, dient als **Ausstiegsbedingung**.
- **Recursive Case:** wiederholt so lange die Funktion, **bis** der **Base Case** erreicht ist.

Challenge 43

Nutze **Rekursion**, um eine **Lösung** für das folgende Problem zu **implementieren**:

Teste ein Wort auf ein Palindrom (das gleiche Wort vorwärts, wie rückwärts).

Deine Eingabe soll ein Wort, die Ausgabe ein Boolean sein.

Beispiele: level, civic, rotor, Lagerregal, Neffen

Hier kommt deine Lösung hin

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel A: String mit *format()*

format()

Mit *format()* können wir, **ähnlich** wie mit **f-Strings**, **Strings und Ausgaben** mit **Inhalten und Variablen** füllen.

Challenge 44

Setze die folgenden **f-Strings** mit *format()* um:

num1 = 21

num2 = 5.5

num3 = 12

```
print(f"Der Preis liegt bei {num3:.2f}€.")
```

```
print(f"Heute um {num1} Uhr ist Ende.")
```

```
print(f"Die Fahrt dauert ca. {num2}h, dann sind wir gegen {num3} Uhr da.")
```

Der Preis liegt bei 12.00€.
Heute um 21 Uhr ist Ende.
Die Fahrt dauert ca. 5.5h, dann sind wir gegen 12 Uhr da.

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel B: Objekte kopieren

Shallow Copy

Python **speichert** jedes **innerste Element** als solches **ab**.
Eine **Liste innerhalb** einer anderen **Liste**, wird also wiederrum **als eigene Liste abgespeichert**, **unabhängig** von ihrer **Verschachtelung in einer anderen Liste**.
Wenn wir mit **`/:`**, **`.copy()`** oder **`copy(X)`** die **äußere Liste kopieren**, **kopieren** wir dabei auch die **Pointer auf ihre inneren Listen** (oder Dictionaries, Sets, etc.).

Deep Copy

Nur mit einer **Deep Copy kopieren** wir die **tatsächlichen Inhalte** aller Elemente.
Hier ist es egal, wie weit unsere Verschachtelungen reichen: **jeder Pointer** wird **in** seine einzelnen **Inhalte aufgelöst**.

Challenge 45

Wir haben ein **Dictionary mit Listen** von **Kunden** gegeben.
Bevor wir Personen oder Firmen **entfernen**, wollen wir eine **Kopie als Archiv** unserer aktuellen Kund*innen **anlegen**.

Erstelle eine Archiv-Kopie und entferne "Mark Crawford" aus der Liste der Privatkunden.
Gib danach sowohl das Archiv, als auch die neue Liste mit korrekten Labeln aus.

```
customers = {"Private": ["Jesse Springer", "Mark Crawford", "Delilah Baron"],  
            "Business": ["Hewett Technologies", "Frontier Global"]}
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel C: Comprehensions

List Comprehension

Mit **List Comprehension** können wir **sehr kompakt Listen** anhand von gegebenen Regeln **erstellen**.

Dazu nutzen wir **"in-line" for-Loops** und können zusätzlich **auch Bedingungen** (ternäre Operatoren) formulieren.

Challenge 46

Nutze **List-Comprehension**, um die folgende **Liste** zu **säubern**:

- Alle Namen sollen mit korrekter Großschreibung in eine neue Liste übertragen werden.
- Nur Namen sollen übernommen werden, alle anderen Werte werden ignoriert.

```
name_list = ["frank", 188, "Jeremy", 174, "tyler", 193, "Rosa", 185, "amy",  
            159, "Carl", 184, "charlotte", 162]
```

Lösung

Set & Dict Comprehension

Wie bei Listen können wir **ebenfalls Sets** und **Dictionaries** mit **Comprehension** erstellen.

Sets verhalten sich hierbei **genau wie Listen**.

Dictionaries benötigen **jeweils ein key/value pair innerhalb** der **Comprehension**.

Challenge 47

Mit **Set- & Dict-Comprehension**, erfülle folgende **Anforderungen**:

- Wir haben eine Liste mit Tupeln, jeweils mit einer Prozess-ID (4-stellige Zahl) und einem Efficiency-Score (2-stellig mit einer Nachkommastelle).
- Wir brauchen ein Set mit nur den Prozess-IDs, welche mindestens einen Efficiency-Score von 70 haben.

- Die Prozesse, welche nicht mindestens einen Score von 70 erreichen, sollen in ein Dictionary mit ihrer ID und ihrem Score gespeichert werden.

```
from random import randint
efficiency_score = [(randint(1000, 9999), randint(10, 1000)/10) for _ in
                    range(100)]
print(efficiency_score)

[(7204, 25.9), (9800, 26.3), (1434, 76.1), (8607, 52.2), (3383, 22.2), (9353,
55.9), (8117, 88.1), (7870, 30.1), (2004, 37.2), (3127, 22.6), (1573, 75.8),
(6422, 40.2), (4411, 100.0), (5215, 27.5), (5587, 95.4), (1710, 84.4), (1535,
39.7), (1063, 42.9), (8601, 29.4), (7712, 64.1), (4910, 13.4), (1388, 68.8),
(2436, 64.4), (7505, 71.2), (8026, 52.7), (5940, 31.2), (2535, 66.9), (9333,
93.1), (6402, 77.4), (7453, 15.4), (1524, 22.4), (8580, 63.9), (7825, 58.3),
(1112, 61.5), (1797, 27.0), (8017, 10.4), (4201, 72.2), (6872, 78.8), (1975,
63.4), (8952, 83.8), (6066, 6.4), (1506, 6.8), (3133, 78.1), (4462, 28.3),
(3589, 61.5), (3561, 38.2), (3771, 33.9), (5906, 26.7), (3242, 45.4), (6870,
79.2), (9136, 11.7), (9250, 92.4), (2968, 72.7), (5719, 73.2), (7340, 1.4),
(3260, 96.6), (1093, 57.7), (1205, 69.6), (2239, 63.3), (7096, 88.9), (5103,
66.0), (8319, 37.0), (9194, 91.9), (4409, 42.3), (7780, 62.4), (9987, 7.1),
(7852, 11.8), (2116, 30.1), (5090, 77.6), (9444, 89.3), (3634, 56.1), (7062,
53.2), (3122, 60.7), (3127, 65.6), (4825, 98.4), (2812, 9.0), (2471, 38.6),
(9071, 6.2), (2377, 99.7), (7554, 6.8), (5040, 48.9), (8656, 36.6), (5657,
83.7), (1476, 51.3), (5718, 27.1), (3041, 7.1), (2502, 67.3), (5565, 93.3),
(5371, 96.6), (2019, 41.2), (3664, 87.4), (5342, 52.4), (9708, 73.3), (1939,
66.0), (1562, 3.9), (5297, 86.1), (8318, 52.4), (9035, 9.9), (2725, 48.7),
(8083, 13.7)]
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel D: Exceptions

Error-Typen

Viele **verschiedene Fehler** können zu vielen **verschiedenen Errors/Exceptions** führen. Davon treten **einige deutlich häufiger** auf, als andere.

Alle **built-in Error-Types** in Python:

https://www.w3schools.com/python/python_ref_exceptions.asp

Raise

Mit **raise** können wir **Fehlermeldungen selber erzeugen** und (bestenfalls) **deskriptive Beschreibungen** ausgeben.

Dies ist **nützlich**, wenn für den korrekten Verlauf des Programm **manche Werte unzulässig** sind.

Meistens ist das zum **Abfangen** von bestimmtem **Nutzer-Input** verwendet, kann aber auch zum **Debugging** oder **Absichern** innerhalb von Programmen dienen.

Challenge 48

Nutze **raise**, um ein **ungewolltes Ergebnis** abzufangen und eine **deskriptive Meldung** auszugeben.

```
from random import randint
```

```
num = randint(-5, 5)
result = num**0.5
print(result)
```

```
1.7320508075688772
```

Lösung

Try & Except

Mit **try** wird ein **Block** von Code **getestet**.

1. **Wenn** dieser **problemlos** durchläuft, läuft auch das **Programm weiter**.
2. **Wenn** es dabei zu **Fehlern** kommt, greif der **except Block**, entsprechend des spezifizierten Fehlertyps.
3. **else** wird bei **fehlerlosem Durchlauf** ausgeführt.
4. **finally** wird **immer** ausgeführt.

Achtung: **Syntax Errors** im Try-Block werden **nicht abgefangen** und führen trotzdem zum kompletten **Abbruch**.

Hierarchie von Exceptions

Challenge 49

Nutze **try** und **except**, um im folgenden Code **potentielle Fehler abzufangen** und **deskriptive Fehlermeldungen** auszugeben:

Verändere nichts am vorhandenen Code!

```
from random import randint
```

```
my_input = randint(0, 10)
result = 0
```

```

for i in range(1, randint(2, 10)):
    result = result + my_input // 2
    result = result / my_input
    my_input -= i

print(f"Result: {round(result, 4)}")

```

Lösung

Assert

Mit **assert** können wir überprüfen, ob eine **Bedingung** an einem **bestimmten Punkt** im Programm einen **erfüllt** ist.

Trifft diese Assertion zu, wird der **Code weiter ausgeführt**, **andernfalls** wird eine **Fehlermeldung** ausgegeben und abgebrochen.

Dies **eignet sich besonders** zum **debuggen** und **absichern** größerer/komplexerer Programme.

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel E: In/Out, mit Dateien arbeiten

Dateien öffnen

Python gibt uns **mehrere Möglichkeiten**, wie wir eine **Datei öffnen** können. Abhängig vom **angegeben Modus**, haben wir **verschiedene Möglichkeiten**, mit der Datei **zu interagieren**.

Die **generelle Syntax** zum Öffnen von Dateien:

open("DRIVE: \PATHNAME.TYPE")

Der **Pfad kann ausgelassen werden**, wenn sich die Datei **im gleichen Ordner** wie das Programm, dass auf sie zugreift, befindet.

Code	Default	Bedeutung	Beschreibung	Erstellt Datei?	--- ----- --- --- ---
open(DATEI, "r")	YES	read	öffnet die DATEI und liest diese.	NO, Error wenn sie nicht existiert.	
open(DATEI, "a")	NO	append	öffnet die DATEI und hängt an diese an.	YES	
open(DATEI, "w")	NO	write	öffnet die DATEI und schreibt		

in diese. | YES | | *open*(*DATEI*, "x") | NO | create | erstellt die *DATEI*. | YES, Error wenn sie existiert. | | *open*(*DATEI*, "+") | NO | and | öffnet die *DATEI* zusätzlich auch zum Lesen oder Schreiben. | Abhängig vom Basis-Modus, in dem geöffnet wird. |

Code	Default	Bedeutung	Beschreibung
<i>open</i> (<i>DATEI</i> , <i>t</i>)	YES	text	öffnet die Datei im Text-Modus.
<i>open</i> (<i>DATEI</i> , <i>b</i>)	NO	binary	öffnet die Datei im Binary-Modus (z.B. Bilder).

Öffnen & Schließen mit *with*

Mit *with* können wir eine Datei öffnen, eingeschobenen Code darunter ausführen, und die Datei automatisch schließen, sobald dieser Code ausgeführt ist.

Dateien lesen

Mit *read* können wir eine Datei öffnen, eingeschobenen Code darunter ausführen, und die Datei automatisch schließen, sobald dieser Code ausgeführt ist.

Wenn wir uns in einer Datei bewegen, bewegen wir auch den Cursor.

Um zu einem bestimmten Punkt in der Datei zu gelangen, benutzen wir *seek*(*X*), wobei *X* der Index aller Zeichen in der Datei ist.

Code	Beschreibung
<i>read</i> ()	liest die gesamte Datei.
<i>read</i> (<i>X</i>)	liest die nächsten <i>X</i> -1 Zeichen.
<i>readline</i> ()	liest die aktuelle Zeile hinter der Cursor Position.
<i>readlines</i> (<i>X</i>)	liest <i>X</i> Zeilen hinter der Cursor Position und speichert jede Zeile als Listenelement.
<i>seek</i> (<i>X</i>)	setzt den Cursor vor das Zeichen mit Index <i>X</i> .

Dateien bearbeiten

Mit *write* können wir Inhalt in eine Datei schreiben.

Je nach Open-Modus wird der Inhalt entweder am Ende angehängen (append) oder überschrieben (write).

Code	Beschreibung
<code>write(TEXT)</code>	schreibt den TEXT hinter den Cursor.
<code>writelines(LIST)</code>	schreibt die Elemente der LIST zusammenhängend hinter den Cursor.

Challenge 50

Setze den folgenden Ablauf mit **I/O-Operationen** um:

1. Die Inhalte von Sample2.txt sollen ausgelesen werden.
2. Jede Zahl (also jede Zeile) soll ein Element in der Liste sein.
3. Die Liste soll aufsteigend sortiert werden.
4. Diese Liste soll dann in einer neuen Datei (z.B. Sample3.txt) gespeichert werden.
5. Diese neue Datei soll zur Kontrolle einmal ausgelesen werden.

Hier kommt deine Lösung hin

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 5: Dorf

Kapitel F: OS, System und Konsole

Sys & OS

Innerhalb von Python können wir mit den **sys** und **os** Modulen viele Informationen ausgeben.

Gewisse **administrative Vorgänge** können wir auch **direkt abbilden**.

Python in der Konsole

Wir können **Python Programme** auch **direkt in der Konsole** ausführen. Dazu müssen wir **in den Ordner navigieren**, in dem die Python Datei liegt und diese **mit Python ausführen**.

```
cd "C:\Users\niklas.nitsch\Documents\Kurse\My Stuff\Py's  
Adventure\Welt_5_Dorf\5_Files"
```

Challenge 51

1. **Erstelle** eine **Python Datei** im Ordner "5_Files".
2. **Schreibe** ein ganz **simples Programm** was **einen Input** annimmt.
3. **Navigiere in diesen Ordner** innerhalb des **Command Prompts**.
4. **Führe das Programm aus** und **übergib** die nötigen **Argumente**.

Programmbeispiel:

```
n = int(input())
print("n" * n)
```

Hier steht der nötige Basiscode für die Aufgabe

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 6: Tempel

Kapitel A: OOP Basics

Klassen & Objekte

Fast alles in Python wird als **Objekt** gespeichert und verarbeitet.

Klassen sind **Vorlagen** bzw. **Entwürfe** für **Objekte**; wir können also **beliebig viele** verschiedene **Objekte** mit den **gleichen Grundrissen** erstellen.

Mit **festgelegten Attributen** haben **Klassen** **keinen** großen **Mehrwert**.

Daher benutzen wir ***** __init__** **um** **Objekte basierend auf einer Klasse mit** gewünschten **Attributen zu erstellen**.

__init__ * **wird immer** ausgeführt, **wenn ein Objekt** auf Basis **einer Klasse** erstellt** wird.

Challenge 52

Erstelle eine **Klasse** als **Grundlage** für **Objekt** des Typs **"Tisch"**.
Erzeuge mindestens zwei **Tische** und **gebe ihre Werte aus**.

Tipp: was ist **notwendig** zu wissen, damit wir uns einen Tisch **vorstellen können** und wie können wir **verschiedene** Tische **unterscheiden**?

Hier kommt deine Lösung hin

Lösung

Objekte verändern

Fast alles in Python wird als **Objekt** gespeichert und verarbeitet.

Klassen sind **Vorlagen** bzw. **Entwürfe** für **Objekte**; wir können also **beliebig viele** verschiedene **Objekte** mit den **gleichen Grundrissen** erstellen.

Playground

Hier hast du Platz, dich auszutoben

Welt 6: Tempel

Kapitel B: Klassen & Methoden

Klassen als Strings

Mit ***** __str__(self)** können wir festlegen, was ausgegeben wird, wenn ein Objekt unserer Klasse als String angesprochen wird.

Dies passiert z.B., wenn wir das Objekt als Ganzes mit `print()` ausgeben.

Challenge 53

Füge deiner **Tisch Klasse** aus der letzten **Challenge 52** eine **String Methode** hinzu.

Hier kommt deine Lösung hin

Copy/paste deinen Code aus der letzten Challenge

Lösung

Methoden in Klassen

Mit ***** __str__()** können wir festlegen, was ausgegeben wird, wenn ein Objekt unserer Klasse als String angesprochen wird.

Dies passiert z.B., wenn wir das Objekt als Ganzes mit `print()` ausgeben.

Challenge 54

Füge zu deiner **Tisch Klasse** aus der letzten **Challenge 53** eine **String Methode** hinzu.

Hier kommt deine Lösung hin
Copy/paste deinen Code aus der Letzten Challenge

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 6: Tempel

Kapitel C: Getter, Setter, Deleter

Getter, Setter & Deleter

Entgegen zu z.B. Java ist in **Python** grundsätzlich **alles public**.
Das heißt, dass wir **auf Variablen** (auch von Klassen/Objekten) grundsätzlich **immer zugreifen** können.

Für **Klassen** sollten viele **Attribute** jedoch **bestimmte Formen** haben und **nicht frei überschrieben** werden können.

Dafür gibt es **Getter**, **Setter** & **Deleter**, in Python mit **@property** (Getter), **@VARIABLE.setter** und **@VARIABLE.deleter**.

Challenge 55

Implementiere **Getter**, **Setter** und **Deleter** für die **Tisch Klasse** aus den vorherigen **Challenges (52 - 54)**.

Überlege hierbei **welche Attribute** **essentiell** sind, welche **bestimmte Typen** haben müssen, was passiert wenn **Attribute gelöscht** werden, etc.

Teste dies mit einem **Objekt** (ausgeben, überschreiben, löschen, erneut ausgeben).

Hier kommt deine Lösung hin
Copy/paste deinen Code aus der Letzten Challenge

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 6: Tempel

Kapitel D: Vererbung von Klassen

Einbindung mit *import*

Ähnlich **wie in-built**s und zusätzliche **Pakete** können wir unsere **eigenen Klassen** (und generell Dateien) mit *import* einbinden.

Vererbung & Child-Klassen

Eine **Klasse (parent)** kann auch **Unterklassen (child)** haben, welche Eigenschaften, Methoden, etc. von ihr **erben**.

Child-Klassen könne beliebig angepasst und erweitert werden.

Es gibt **verschiedene Herangehensweisen** zur Vererbung.

```
***PARENT.__init__(self, PARAMETERS)***
```

- basiert **genau** auf der **Klasse**, die **angegeben** wird
- kann **Schritte** der **MRO überspringen**
- kann Klassen **mehrfach initialisieren**

```
***super().__init__(PARAMETERS)***
```

- generell der **"good practice approach"**
- genaue **Vererbung** basiert auf Python's **MRO**
- **keine Dopplungen** von Initialisierungen

Challenge 56

Implementiere deine **bisherige Tisch Klasse** als **Unterklasse** der Klasse **Moebel** (in 6_Files).

Beachte dabei folgendes:

1. Um Vererbung, Auslagerung und Import korrekt abzubilden, müssen wir getrennte Python Dateien verwenden (siehe 6_Files).
2. Lagere so viel wie möglich in die Parent-Klasse aus, was (fast) alle Möbel gemeinsam haben.
3. Füge extra Attribute hinzu, die nur für deine Tisch Klasse relevant sind.
4. Achte auf korrekte Setter, Getter & Deleter und passe diese so an, dass sie sinnvolle Validierungen machen.

Teste deine Klassenstruktur mit **mindestens zwei Objekten** der Klasse **Tisch**.

Lösung

Die Dateien mit Musterlösungen bekommst du nach der Aufgabe

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel A: Short Circuit Evaluation

Short Circuit Evaluation

Wenn bei einer Verkettung von Vergleichen **ein Fall zutrifft**, werden die **restlichen Bedingungen ignoriert**.

Wenn ein **Fall** grundsätzlich **immer zutrifft**, wird dies auch **im Code reflektiert**.

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel B: Daten mit *datetime*

datetime Basics

Das **in-built** Modul *datetime* ermöglicht uns einen **vielfältigen Umgang mit Daten**.
Damit können wir z.B. **Daten formatieren**, **Zeiten messen** und uns alle **nötigen Informationen** aus einem **vollständigen Datum-Objekt** asugeben lassen.

Challenge 57

Mit Hilfe von *datetime*, bestimme die **Zeit**, die **vergangen ist**, seit:

1. dem 10.12.2012 in Tagen
2. heute morgen, 7:45 in Stunden, Minuten & Sekunden
3. dem 01.01.2025, 0:00, in Tagen, Stunden, Minuten & Sekunden

Hier kommt deine Lösung an

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel C: Laufzeit messen

Laufzeit

Um unsere Programme zu **optimieren**, müssen wir wissen, **wie lange** sie für eine bestimmte Operation **brauchen**.

Dafür **messen** wir die gesamte **Laufzeit** und **vergleichen** verschiedene Herangehensweise bzw. **Optimierungen**.

Es gibt viele **verschiedene Möglichkeiten**, um Laufzeiten zu messen.

Challenge 58

Wähle eine der **vorherigen Challenges** oder **Skill-Checks**:

z.B. **Fibonacci** (rekursiv vs. while/for), **Happy Numbers**, **Sortierungs-Algorithmen**, etc.

BONUS: Lasse dir die **Laufzeiten als Plot** anzeigen (mit **matplotlib**).

1. Wenn noch nicht vorhanden, implementiere eine alternative Herangehensweise (z.B. while statt for-Loop).
2. Messe die Laufzeiten der beiden Programme über mehrere Durchläufe (z.B. 1000).
3. Vergleiche die durchschnittliche Laufzeit.

*# Entweder copy/paste hier deinen Code, oder nutze dafür eigene Python
Dateien (in 7_Files)*

Version 1

*# Entweder copy/paste hier deinen Code, oder nutze dafür eigene Python
Dateien (in 7_Files)*

Version 2

Big-O Notation

Die **Runtime** eines **Programms** wird **von** der mathematischen **aymptoten Notation** abgeleitet.

$O(N)$

wo **O** für "**order of operations**" (Anzahl an Operationen) und **N** für die **Anzahl an verarbeiteten Elementen** steht.

$O(N^2)$

für **jedes Element**, das verarbeitet wird, **erhöht** sich die **Laufzeit quadratisch**.

Runtime Optimierung in der Mathematik:

<https://www.youtube.com/watch?v=KzT9IId-LlQ>

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel D: Generatoren

Generator Expressions

Generatoren ermöglichen das **effiziente Erstellen** von **großen Datenmengen**.

Durch einen Generator wird **erst dann eine Teilmenge** seiner Daten erzeugt und gespeichert, **wenn dieses auch gebraucht** werden.

Dabei wird das **verwendete Element "aufgebraucht"** und **aus dem Generator entfernt** und nicht mehr Speicher benötigt.

Dies wird **besonders relevant**, wenn du **größere, komplexere Programme** implementierst, oder mit **sehr großen Datenmengen** arbeitest.

Challenge 59

Nutze einen **Generator**, um die folgende **Anforderung** zu **erfüllen**:

Eine Liste mit Tupeln, wobei jedes Tupel eine Zahl und ihre Quadratwurzel beinhaltet;

für alle Ganzzahlen bis 1000.
Gib dann die ersten 25 Zahlen mit ihren Quadratwurzeln aus.

Hier kommt deine Lösung hin

Lösung

Generator Funktionen

Mit **yield** können wir, im **Gegensatz zu return**, **mehrere Rückgaben** innerhalb **einer Funktion** machen und somit **spezielle Generatoren erzeugen**.

Challenge 60

Nutze einen **Generator**, um die folgende **Anforderung** zu **erfüllen**:

Eine Funktion, die einen Parameter annimmt und alle durch diesen Parameter ganz teilbaren Zahlen abbildet.

Die Funktion soll danach aufgerufen werden um alle dieser Zahlen bis 1000 auszugeben.

Hier steht der nötige Basiscode für die Aufgabe

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel E: Args & Kwargs

***args**

Mit einem Stern ***** **vor Argumenten** können wir einer Funktion **beliebig viele Argumente als Tupel** übergeben.

In Dokumentationen werden arbiträre Argumente meist als ***args** angegeben.

***args = Arguments**

****kwargs**

Mit zwei Sternen **** vor Argumenten** können wir einer Funktion **beliebig viele Keyword-Argumente als Dictionary** übergeben.

In Dokumentationen werden arbiträre Keyword-Argumente meist als ****kwargs** angegeben.

****kwargs = Keyword-Arguments**

Challenge 61

Nutze ***args** um die folgende **Funktion** zu **implementieren**:

Die Funktion soll eine beliebige Anzahl Listen entgegennehmen und diese als eine flache Liste zurückgeben.

Teste mindestens mit: [1, 2, 3], [4], [5, 6], [7, 8, 9]

CHALLENGE: Erweitere die Funktion, sodass sie Listen mit beliebig vielen Unterlisten (nested lists) als eine einzelne Liste zurückgibt.

TIP: Rekursion kann hier helfen.

Teste mindestens mit: [1, 2, 3], [4], [5, 6, [1, 5, [1, 4, [5, 5]]]] und möglichst tiefen Nestings (z.B. 20 Listen tief).

Hier kommt deine Lösung hin

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel F: Map()-Funktion

map()

Mit **map(FUNKTION, ITERABLE)** können wir eine **Funktion** auf **jedes Element** eines **iterierbaren Objekts** (z.B. Liste) **anwenden**.

Dabei wird der Funktion **jeweils** immer ein **Index als Parameter** übergeben.

Dabei **bekommen** wir ein **map-Objekt** zurück, welches wir **als Liste ausgeben** oder **durch** welches wir **iterieren** können.

Wir können einer **Funktion**, die **mehrere Parameter erwartet**, auch **mehrere iterierbare Objekte übergeben**.

Hierbei werden dem **map-Objekt** nur **so viele Rückgaben hinzugefügt**,** wie Parameter im kürzesten** übergebenen **iterierbaren Objekt** sind.

Challenge 62

Nutze **map()** und **Funktionen**, um den folgenden **Code übersichtlicher** und **kürzer** zu machen.

TIP: **Loops** und **append()**s sollten **wegfallen**.

```
numbers = [1, 2, 3, 4, 5]

squared_numbers = []
for n in numbers:
    squared_numbers.append(n ** 2)

string_numbers = []
for n in squared_numbers:
    string_numbers.append(str(n))

print(string_numbers)

['1', '4', '9', '16', '25']
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 7: Katakomben

Kapitel G: Lambda

Lambda

Mit **lambda** können wir **anonyme Funktionen** deklarieren.
Damit können wir **Code kompakter machen**, indem wir z.B. eine **Funktion**, die wiederholt mit **verschiedenen Parametern** genutzt wird, **als Blaupause** schreiben.

Einige **built-in Funktionen**, und auch viele **zusätzliche Module** können **Lambda als Parameter** nutzen.

```
from functools import reduce

my_list = [1, 2, 3, 4]

list_mult = reduce(lambda x, y: x* y, my_list)
print(list_mult)
```

24

Challenge 63

Nutze *lambda* um folgenden **Code** zu **optimieren** und **Redundanzen** zu **vermeiden**:

```
def add_prefix_de(phone_num):
    if phone_num[0] == 0:
        phone_num = "+49" + phone_num[1:]
    else:
        phone_num = "+49" + phone_num
    return phone_num

def add_prefix_gb(phone_num):
    if phone_num[0] == 0:
        phone_num = "+44" + phone_num[1:]
    else:
        phone_num = "+44" + phone_num
    return phone_num

def add_prefix_gb(phone_num):
    if phone_num[0] == 0:
        phone_num = "+1" + phone_num[1:]
    else:
        phone_num = "+1" + phone_num
    return phone_num
```

Lösung

Playground

Hier hast du Platz, dich auszutoben

Welt 8: Projekte

Kapitel A: Projekt

Überblick & Themen

Jetzt wollen wir unser Python-Wissen **praktisch umsetzen**.

Dazu eignen sich **größere Projekte**, an denen wir über mehrere Einheiten arbeiten können.

Themenideen:

- Home-Office Calculator
 - MOTD Ausgabe (mit/ohne API)
 - API Anbindung (mit/ohne OOP)
 - Text-Adventure
 - Snake in Python
 - ...
-

DocString

1. Was ist ein DocString?

Wird **vom Computer** (Compiler/Interpreter) **als Beschreibung** der Funktion/Klasse hinterlegt.

Dient zur **Dokumentation** von **Funktionen**, **Methoden** und **Klassen**.

2. Warum sollte ich DocStrings nutzen?

Wir können unsere **eigenen** (komplexeren) **Funktionen**, **Methoden** & **Klassen** für andere Nutzer*innen **klar beschreiben**.

Benötigte, sowie optionale **Parameter**, **Rückgaben** und generelle **Funktionalität** können **formatiert dargestellt** werden (abhängig von der IDE).

```
def sum_function(a, b):  
    # DocString  
    """  
    :param a: first summand  
    :param b: second summand  
    :return: sum of a & b  
    """  
    return a + b
```

Welt 8: Schatzkammer

Kapitel B: GUI mit tkinter

tkinter Basics

Mit **tkinter** können wir direkt in Python **simple GUIs** erstellen.
Die **Funktionalität** und **Design-Möglichkeiten** sind recht **limitiert**, aber bieten eine **gute Grundlage** zum erstellen von GUIs für eure Projekte.

tkinter Dokumentation:

<https://docs.python.org/3/library/tkinter.html>

tkinter Grundlagen:

<https://www.geeksforgeeks.org/python/python-gui-tkinter/>

Welt 8: Schatzkammer

Kapitel C: Websites in Python mit flask

flask Basics

Mit **flask** können wir in Python **Websites** erzeugen und **lokal hosten**.
So können wir unsere **Anwendungen** auch direkt **als Website ausgeben** und gestalten.
Zum **Aufbau** benötigt man jedoch Grundkenntnisse von **HTML**, zum **Styling CSS** und für erweiterte **Interaktionen JavaScript**.

flask Dokumentation:

<https://flask.palletsprojects.com/en/stable/>

flask Grundlagen:

<https://www.geeksforgeeks.org/python/flask-tutorial/>

Die Realität von JavaScript:

<https://www.youtube.com/watch?v=aWfYXg-Ypm4>